



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校 河海大学

参赛队号 24102940057

1. 罗子杰

队员姓名 2. 庞榕泽

3. 赵勇凯

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： 考虑机组实时疲劳损伤的风电场有功功率优化分配

摘 要：

本文将复杂的考虑机组实时疲劳损伤的风电场有功功率优化分配问题，拆解为多个子问题，通过建立累积损伤指标实时计算模型、载荷-参考功率拟合模型实现将实时疲劳损伤纳入优化分配模型，采用强化学习求解风电场有功功率分配，最终取得全局较优解。

对于问题一，针对雨流计数法难以实时在线应用的局限性，本文提出**实时雨流计数法**，相较于传统雨流计数法，所提算法在确保**计算精度不变**的基础上，实现载荷循环的**实时与高效计算**。针对实时雨流计数法在时间末端才能最终确定最大载荷循环，从而延迟了对关键时段巨大载荷波动的实时响应，以及线性累积疲劳损伤模型无法考虑载荷循环顺序效应的问题，本文进一步提出**实时非线性累积疲劳损伤指标**，所提指标通过引入时间修正系数与暂时损伤，实现有效考虑载荷循环的顺序效应，实时识别最大载荷循环，**实现对风机损伤关键时段巨大载荷波动的实时识别与累积疲劳损伤的精准评估**。算例计算结果显示，模型求解快速准确，能够实时识别出风机载荷大幅波动时刻及不同风机损伤分布情况：计算耗时 **0.739 秒**，**87 号** 风机主轴在第 **27 秒** 出现载荷大扰动，主轴损伤量级最大；**7 号** 风机塔架第 **18 秒** 出现载荷大扰动；**14 号** 风机塔架第 **25 秒** 出现载荷大扰动；**40 号** 风机塔架第 **1 秒** 出现载荷大扰动，到第 **16 秒** 才逐渐结束，塔架损伤量级最大；**73 号** 风机塔架第 **80 秒** 出现载荷大扰动；**75 号** 风机塔架第 **86 秒** 出现载荷大扰动；**81 号** 风机塔架第 **36 秒** 出现载荷大扰动；**95 号** 风机塔架第 **95 秒** 出现载荷大扰动。

对于问题二，针对塔架推力和主轴扭矩通常无法直接通过测量获得的问题，本文深入剖析了风机运行中的复杂物理过程，包括输出功率响应的滞后性、风速与载荷变化的相互作用，构建**风速与参考功率的耦合关系模型**，通过物理机理分析、最小二乘拟合，提出**高效准确的载荷-参考功率拟合模型**。利用附件 2 提供数据，通过迭代优化算法求解模型参数。经测试，本文提出的载荷-参考功率预测模型能够精确识别功率指令和风速与机械应力间的数值关联，对于主轴扭矩识别平均误差 **6%**，塔架推力识别平均误差 **20%**。

对于问题三，针对传统算法在实时求解效率与全局优化能力上的不足，本文构建基于**强化学习**的风电场功率分配模型，能够**高效解算**动态高维场景，同时融合历史载荷特征、当前风速信息及未来功率需求预测，实现**风电场功率分配的实时优化**。针对神经网络拟合产生的功率越限问题，本文通过构造**特殊的神经网络层和放缩限幅策略**，确保功率分配严格符合系统约束，并通过安全层对极少数功率越限情况进行校正。通过实时求解功率分配模型，计算得到功率分配策略，实现部分风机的累计疲劳损伤显著降低，**总体累积疲劳损**

伤降低 10%，载荷波动方差减小 30%。

对于问题四，针对风电场环境数据与负荷需求信息的不确定性及噪声干扰问题，本文采取了双重策略应对。通过设定判断条件识别数据阻塞现象，并引入预测算法将不确定性转化为预测精度问题。构建**鲁棒优化模型**，通过迭代求解最差场景下的最优策略，确保模型在多数有效场景下既能满足所有约束条件，又能保持一定的优化效能，为风电场在复杂多变环境下提供了更为稳健与保守的功率分配方案。

关键词： 实时雨流计数法；非线性累积疲劳损伤；强化学习；鲁棒优化；

一、问题重述

1.1 问题背景

近年来，风电迅速发展，规模风电场建成使用。风机发电过程中，机械部分将出现疲劳损伤，且随发电功率提高而出现加速疲劳，造成不良影响。因此，有必要对其进行优化配置，以提高运行中的经济性、安全性。

风电站对电网调度指令 P_t 进行集中响应，即指令发送场站后，其将按照一定的分配方案计算每台风机的出力 P_{ref}^i ，各风机按照 P_{ref}^i 控制自身参数，从而实现实际出力 P_{out} 对 P_{ref} 的跟踪。传统分配为平均分配，即各风电机组分配出力 $P_{ref}^i = P_t / W_t$ ， W_t 为站内风机台数。 P_{ref}^i 受当前风速限制，但该方案未考虑风机机械部分的疲劳，存在不足。

1.2 问题提出

问题一：利用主轴扭矩、塔架推力对风机主轴、塔架累积疲劳损伤程度进行量化，在保证量化正确性的前提下降低计算的复杂性，实现任意时刻的实时计算。

问题二：建立主轴扭矩、塔架推力与风速、参考功率的关系模型，实现从风速、风机参考功率到主轴扭矩、塔架推力的估计。

问题三：考虑主轴、塔架疲劳损伤下，建立站内风机实时功率分配模型，采用累积疲劳损伤程度作为目标，对比参考功率进行优化前后的数据。

问题四：由于实际情况中，存在测量误差、数据采集传输过程通信延迟影响，需对前述模型进行修正。

二、问题分析

2.1 问题一分析

问题一需要对任意时段累积疲劳损伤程度量化指标进行实时计算，主要内容分为两部分，包括提出能够尽可能保证精确性的实时计算算法，以及提出能够克服实时计算缺陷并保证量化效果的量化指标。

从实时计算算法方面看，需要克服的难点在于如何解决常规算法不能实时计算的缺点，提高实时计算的效率与准确性，并且能够尽可能减小实时计算算法自身的缺陷；从量化指标方面看，需要克服的难点在于如何保证量化指标准确评估疲劳损伤程度，并且能够对关键时刻的数据快速反应，提高实时性。综合而言，实时计算和指标提出需要结合起来分析，需同时考虑二者之间影响，实时计算的缺点可能导致量化指标的高度复杂，量化指标不能反应关键时刻的数据会导致实时计算没有意义。

2.2 问题二分析

问题二需建立风机有功功率参考值、风机风速和风机轴系扭矩、塔顶推力的数学模型，该问题的难点在于风机根据参考功率控制自身输出功率是一个复杂的动作过程，包括控制桨距角、转矩进行调整以及多次的反馈控制过程，同时还受风速的影响，以及动作的时延性，需要直接从复杂物理过程分析变量之间的关系，再通过最小二乘法进一步拟合得到更为准确的数学模型。

2.3 问题三分析

问题三需要风电场根据调度指令、风机状态将指标分解给风电场内各风机，主要内容是实现对风机有功功率分配策略的求解。

从分配策略求解方法方面看，目前主要有以下几类方法：

启发式算法，可以根据风电场风速、调度指令，进行初始化、寻优、收敛，方法简单、易实现，但是其解的最优性难以保证，且依赖初始状态，易陷入局部最优，并且收敛过程较长，实时性较差。

数学优化算法，可以将风电场功率分配问题描述为数学模型，再使用优化方法进行求解，但常规优化大部分为线性优化或将参数线性化，而风机累积疲劳损伤与分配功率之间关系较为复杂，难以线性化，可能导致最终优化结果较差，并且大量风机导致数学模型庞大、维数较高，实时性与精确性都将受到影响。

强化学习方法作为正在兴起的算法，可以有效嵌入变量的不确定性，经过训练的模型兼顾计算速度和优化精度，进而实现实时功率分配。然而，强化学习存在无法充分考虑优化问题的约束条件，需要考虑安全修正以保证约束条件的满足。

2.4 问题四分析

问题四需要建立考虑数据测量误差随机性和传输延时的影响，同时尽可能保证模型的鲁棒性。针对测量误差的随机性，考虑建立基于鲁棒优化的优化模型，从而实现存在测量误差情况下，误差最恶劣情况下的目标函数最优；针对传输延迟，依次使用历史数据进行时序预测，尽可能保证数据的准确性。

三、模型假设

假设 1：问题一只考虑载荷循环的顺序效应，忽略现场环境条件的影响，不考虑载荷间的相互作用以及材料裂纹扩展机制；

假设 2：问题二只考虑风机控制系统的动作延时，不考虑通信延时；认为同一风机运行在高风速区，假设风机机械功率转化为输出功率的效率为常数。

四、问题一：模型建立与求解

针对问题一风机的两种不同元件（主轴和塔架）任意时段累积疲劳损伤程度量化指标的实时计算的要求，本节主要采用**实时雨流计数法（real-time rainflow, RRF）**提取随机载荷谱的载荷循环，提出**实时非线性累积疲劳损伤（real-time nonlinear cumulative fatigue damage, RNCFD）**指标量化任意时段累积疲劳损伤程度，所提指标能够实时准确地反映各个风机损伤分布，下面具体解释说明所提模型指标，并分析问题求解结果。

4.1 实时雨流计数算法

为克服雨流计数法（rainflow, RF）无法实时在线计算的缺陷，本节提出了一种适用于风机和塔架随机载荷谱的实时雨流计数法，本节主要介绍实时雨流计数法的原理，并与传统雨流计数法对比。

4.1.1 RRF 算法原理介绍

本节所提 RRF 算法全部流程包括以下模块：数据预处理模块、算法预存库模块、实时雨流计数模块以及载荷循环存储库模块。

以主轴随机载荷谱为例，详细介绍 RRF 算法，具体流程如下：

1、首先，数据预处理模块，对前一时刻的主轴载荷数据 x_2 进行预处理，结合前二时刻的主轴载荷数据 x_1 以及当前时刻的主轴载荷数据 x_3 判断 x_2 是否为极值点，根据下式判断筛选出极值点数据，当满足下式时， x_2 为极值点，将 x_2 存入算法预存库，若不满足下式，则将 x_2 舍去，进入下一时刻；

$$(x_2 - x_1)(x_2 - x_3) > 0 \quad (4.1)$$

2、然后，算法预存库模块，对算法预存库中的载荷数据进行计数，若小于 3，则进入下一时刻，等待新极值点的输入；若大于等于 3，则将算法预存库中的载荷数据按最新向旧的顺序，依次标记为 W 、 V 、 U ；

3、再者，实时雨流计数模块，计算 U ， V ， W 相邻的差值，即：

$$\begin{cases} k_1 = |V - U| \\ k_2 = |V - W| \end{cases} \quad (4.2)$$

- 比较 k_1 、 k_2 ，若 $k_2 < k_1$ ，说明并未形成载荷循环，进入下一时刻，等待新极值点的输入，重新标记 W 、 V 、 U ；若 $k_2 > k_1$ ，说明形成载荷循环，判断半循环、全循环。
- 对算法预存库中的载荷数据进行计数，若等于 3，则表明此载荷循环为半循环，计数计为 0.5，载荷循环幅值为 $T_a = k_1$ ，载荷循环均值为 $T_m = (U + V)/2$ ，将 U 从算法预存库移出， V 、 W 前移；若不等于 3，则表明此载荷循环为全循环，计数计为 1，载荷循环幅值为 $T_a = k_1$ ，载荷循环均值为 $T_m = (U + V)/2$ ，将 U 、 V 从算法预存库移出， W 前移。
- 将得到的载荷循环数据存入载荷循环存储库，若算法预存库的载荷数据数量仍大于等于 3，继续标记 W ， V ， U ，直至算法预存库中的载荷数据数量小于 3，进入下一时刻，等待新极值点的输入。

为更清晰说明 RRF 算法的流程，RRF 算法流程图如图 4.1 所示。

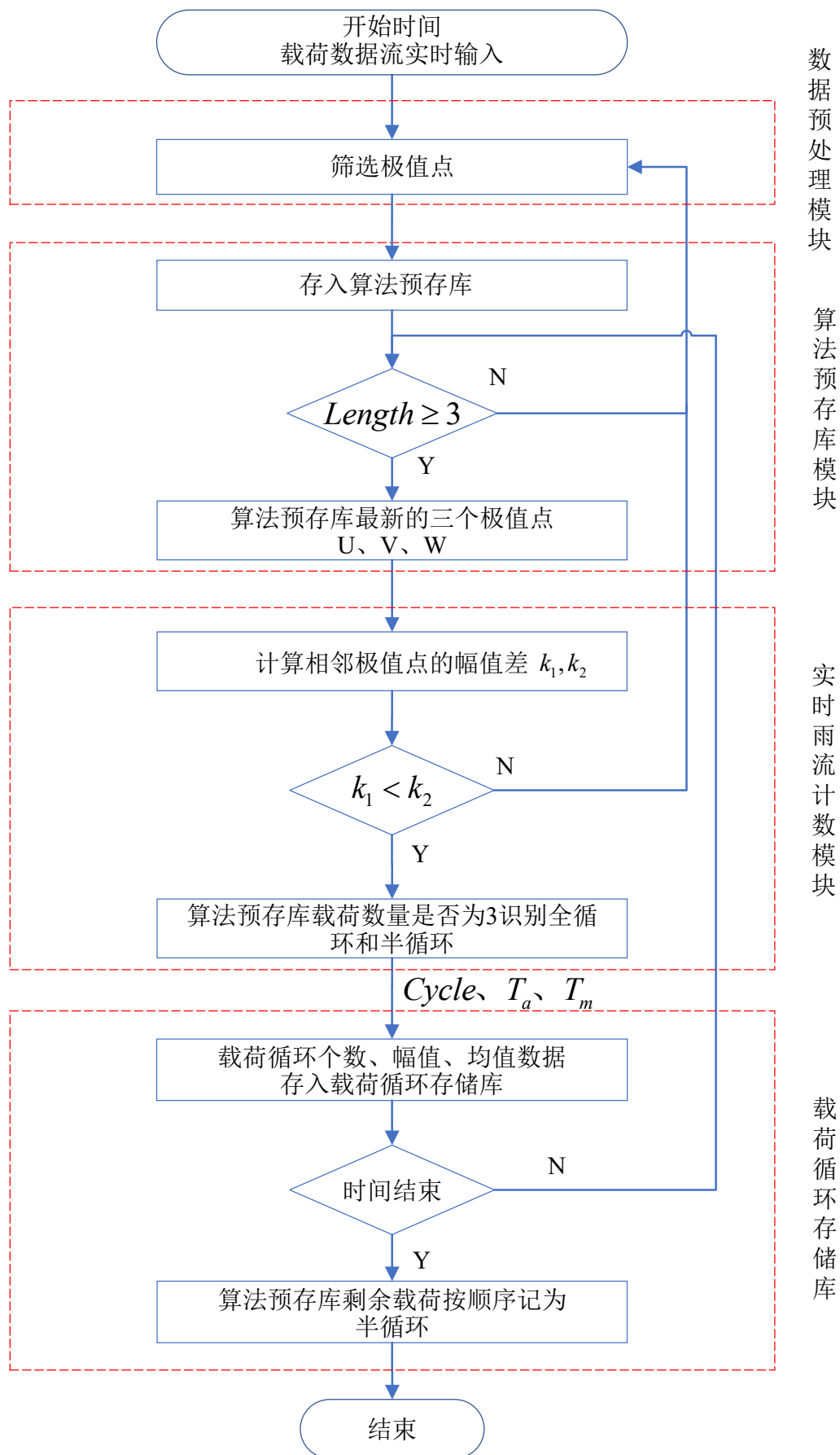


图 4.1 RRF 算法流程图

4.1.2 RRF 算法特点分析

与 RF 算法对比，RF 算法每次计数都需要判断载荷循环起始位置包含于过程 k_1 与否，需要对其进行移动，并且 RF 算法完成所有时刻的载荷循环判断、提取载荷循环后，还需对剩余极值点进行拼接、提取，再拼接、再提取，需要多次循环。

而本文所提 RRF 算法能够实时计算载荷循环，通过直接比较算法预存库的载荷数据数量与 3 的大小来判别全循环、半循环，若 k_1 包含载荷循环起始位置，算法预存库的载荷数据数量为 3；若 k_1 不包含载荷循环起始位置，说明 k_1 前还有极值点，算法预存库的载荷数据数量大于 3，每次从当前时刻的数据开始，实时进行剩余极值点的拼接、提取。

然而，经过上述分析可以发现，RRF 算法在最后时刻才能将最大载荷循环提取出，若最大循环在初始时刻出现，RRF 算法会一直判断到最后时刻直到发现不存在更大的载荷循环时才将最大载荷循环存入载荷循环存储库，导致最大载荷循环的损伤影响在最后时刻才显现，因此 RRF 算法虽然能够实时计算载荷循环，但无法实时考虑最大载荷循环影响。

上述简单分析了 RRF 算法的特点，其具体优越性及缺陷分析，将在第九章进行详细解释说明。

4.2 实时非线性累积疲劳损伤指标

由于线性累积疲劳损伤指标无法考虑载荷循环的顺序效应，导致无法准确评估开始时刻的大载荷循环的损伤影响；以及实时雨流计算法需要计算完所有时刻载荷数据才能提取出最大载荷循环，最大载荷循环的损伤影响在最后时刻才显现，导致累积疲劳损伤指标会在最后时刻指数级突增。为克服上述缺陷，本节提出实时非线性累积疲劳损伤指标，实现实时考虑最大载荷循环的损伤影响，以及考虑载荷循环的顺序效应，本节主要介绍 RNCFD 指标的原理。

4.2.1 RNCFD 指标介绍

对于线性累积疲劳损伤（linear cumulative fatigue damage, LCFD）指标，其常通过对 RF 算法计算随机载荷谱提取出的载荷循环处理得出，其具体表达式如下：

$$LCFD = \sum_{i=1}^m \frac{n_F^i}{N_F^i} \quad (4.3)$$

式中： n_F^i 为每种载荷循环的个数，全循环为 1，半循环为 0.5（相同幅值、均值的载荷循环为同一种载荷循环，个数相加）； N_F^i 为每种载荷循环对应的循环最大承受次数； m 为一段时间计算出的总载荷循环种数。

N_F^i 的具体计算表达式为：

$$N_F^i = \frac{C}{\left(\frac{T_a^i}{T_m^i}\right)^{10} \left(1 - \frac{T_m^i}{\sigma_m}\right)} \quad (4.4)$$

式中： T_a^i 为每种载荷循环的幅值； T_m^i 为每种载荷循环的均值； σ_m 为材料在拉伸断裂时的最大承受载荷，在本题中主轴和塔架两种元件均为 50000000； C 为 S-N 曲线公式的常数项，本题取 9.77×10^{70} 。

然而，LCFD 指标无法考虑载荷循环的顺序效应，并且由于上节采用的 RRF 算法在最后时刻才能提取出最大载荷循环，导致 LCFD 指标既缺乏实时性，也难以准确反映各个风机损伤分布，因此下面对 LCFD 指标进行修正，提出 RNCFD 指标，RNCFD 指标表达式如下：

$$RNCFD(t) = \left(\frac{n_{FM}}{N_{FM}}\right)^{1-f(t)} + \sum_{i=1}^{m(t)} \left(\frac{n_F^i}{N_F^i}\right)^{1-f(t)} \quad (4.5)$$

式中： n_F^i 为实时计算的每种载荷循环的个数，全循环为 1，半循环为 0.5（相同幅值、均值的载荷循环为同一种载荷循环，个数相加）； N_F^i 为实时计算的每种载荷循环对应的循环最大承受次数； $f(t)$ 为每种载荷循环的时间修正系数，具体表达式为 $f(t) = e^{-t/5}/40$ ，时间越靠前，修正系数越大，从而载荷循环影响越大； $m(t)$ 为当前时刻计算出的载荷循环种数，随时间变化而增加。 n_{FM} 为当前时刻算法预存库里最大载荷循环的个数，始终记为半循环 0.5； N_{FM} 为当前时刻算法预存库里最大载荷循环对应的循环最大承受次数。

4.2.2 RNCFD 指标特点分析

分析式 (4.5)，RNCFD 指标分为两部分，前一部分为暂时损伤，是当前时刻 RRF 算法预存库里的最大载荷循环，看作为半循环，实时将其纳入考虑，就不会出现最后时刻才提取出最大载荷循环，导致指标最后时刻出现指数级突增；后一部分为实际损伤，是当前时刻已经提取的载荷循环产生的损伤，通过考虑时间修正系数，将时间靠前的载荷循环影响放大，从而考虑到载荷循环的顺序效应，开始时刻的载荷循环影响大于最后时刻的载荷循环影响。RNCFD 指标适用于风机主轴以及塔架的累积疲劳损伤程度的实时计算。

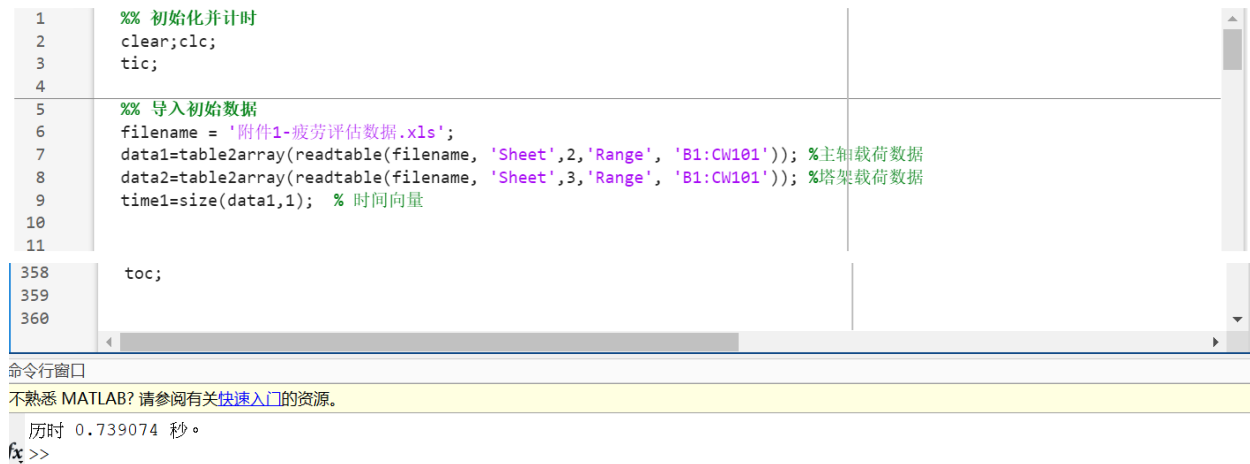
综合考虑最大载荷循环影响以及载荷循环的顺序效应，本文所提 RNCFD 指标能够实现风机的两种不同元件（主轴和塔架）任意时段累积疲劳损伤程度量化指标的实时计算。

4.3 问题求解结果与分析

上两节通过介绍了提取载荷循环的 RRF 算法与根据载荷循环计算的 RNCFD 指标分析各风机任意时段累积疲劳损伤程度，下面将对问题求解结果进行展示并进行分析。

4.3.1 问题求解计算时间分析

首先验证计算时间小于 1.00s 的要求，通过 MATLAB2023b 的 tic、toc 函数，在 13th Gen Intel(R) Core(TM) i9-13900HX 处理器环境下运行算法程序，求得本文所提 RRF 算法与 RNCFD 指标的求解计算时间，其结果见图 4.2。



```

1 %% 初始化并计时
2 clear;clc;
3 tic;
4
5 %% 导入初始数据
6 filename = '附件1-疲劳评估数据.xls';
7 data1=table2array(readtable(filename, 'Sheet',2,'Range', 'B1:CW101')); %主轴载荷数据
8 data2=table2array(readtable(filename, 'Sheet',3,'Range', 'B1:CW101')); %塔架载荷数据
9 time1=size(data1,1); % 时间向量
10
11
358 toc;
359
360

```

命令窗口
不熟悉 MATLAB? 请参阅有关快速入门的资源。
历时 0.739074 秒。
fx >>

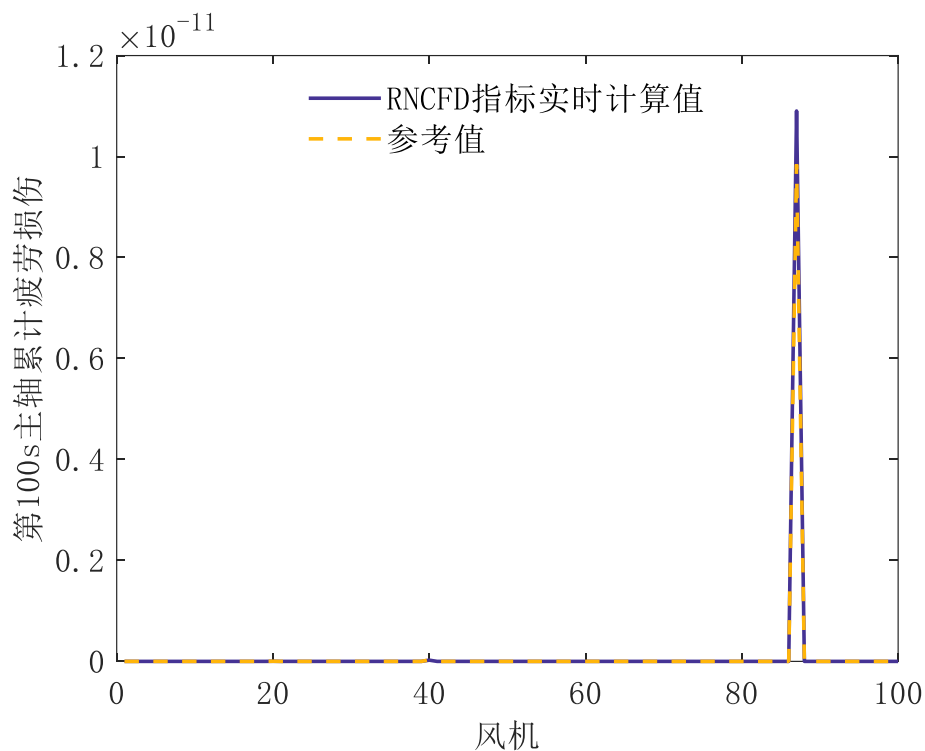
图 4.2 算法程序运行求解时间

根据图 4.2，本文所提 RRF 算法与 RNCFD 指标求解时间为 0.739 秒，满足小于 1 秒的性能要求。

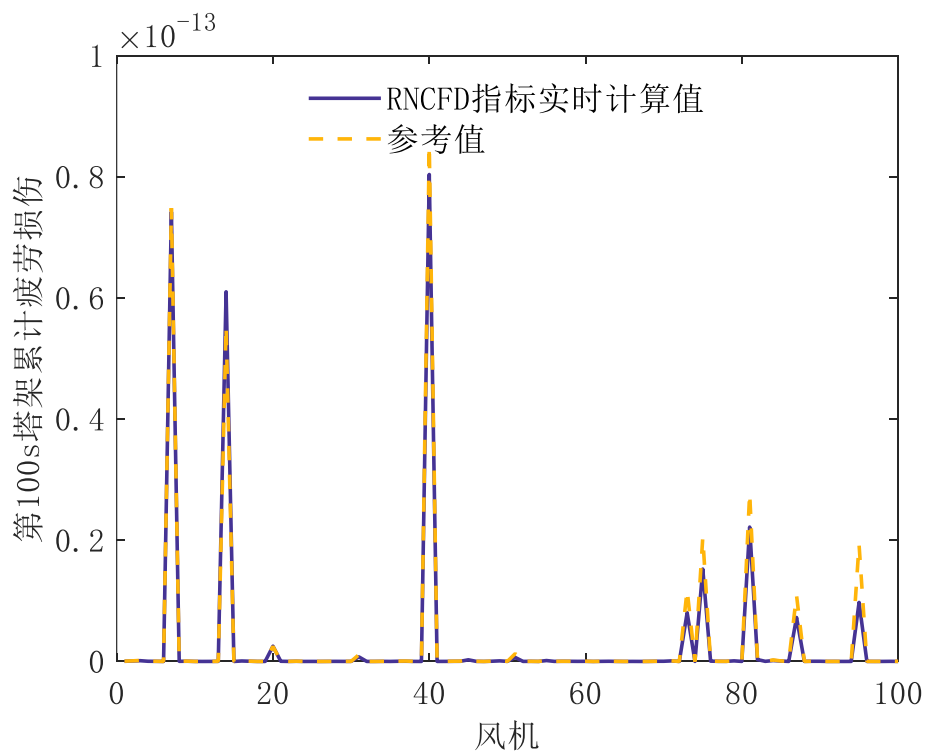
下面将分析问题具体求解结果，并验证 RNCFD 指标的准确性与实时性。

4.3.2 各风机元件第 100s 的累积疲劳损伤分布情况

首先分析各风机元件第 100s 的累积疲劳损伤分布，各风机主轴和塔架第 100s 的 RNCFD 指标分布以及附件 1 中所给参考值分布对比结果如图 4.3（a）、（b）所示。



(a) 各风机主轴第 100s 累积疲劳损伤分布（纵轴量级 10^{-11} ）



(b) 各风机塔架第 100s 累积疲劳损伤分布（纵轴量级 10^{-13} ）

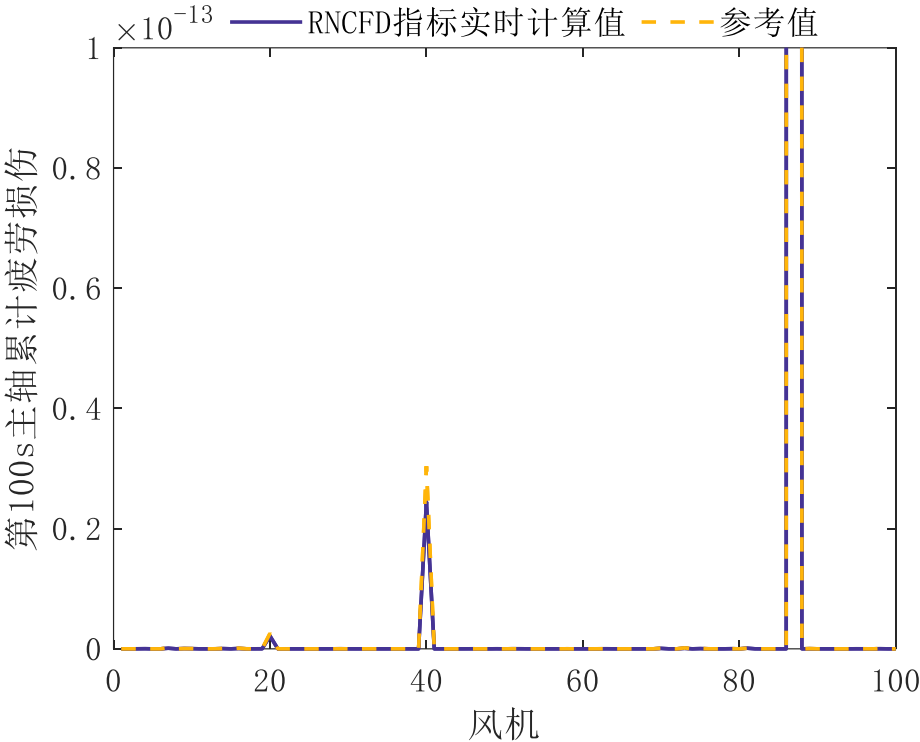
图 4.3 各风机元件第 100s 累积疲劳损伤分布

分析图 4.3 可知，在第 100s 时刻，对于主轴损伤而言，87 号风机的累积疲劳损伤最严重，超过其余风机多个数量级。在第 100s 时刻，对于塔架损伤而言，40 号风机的累积疲

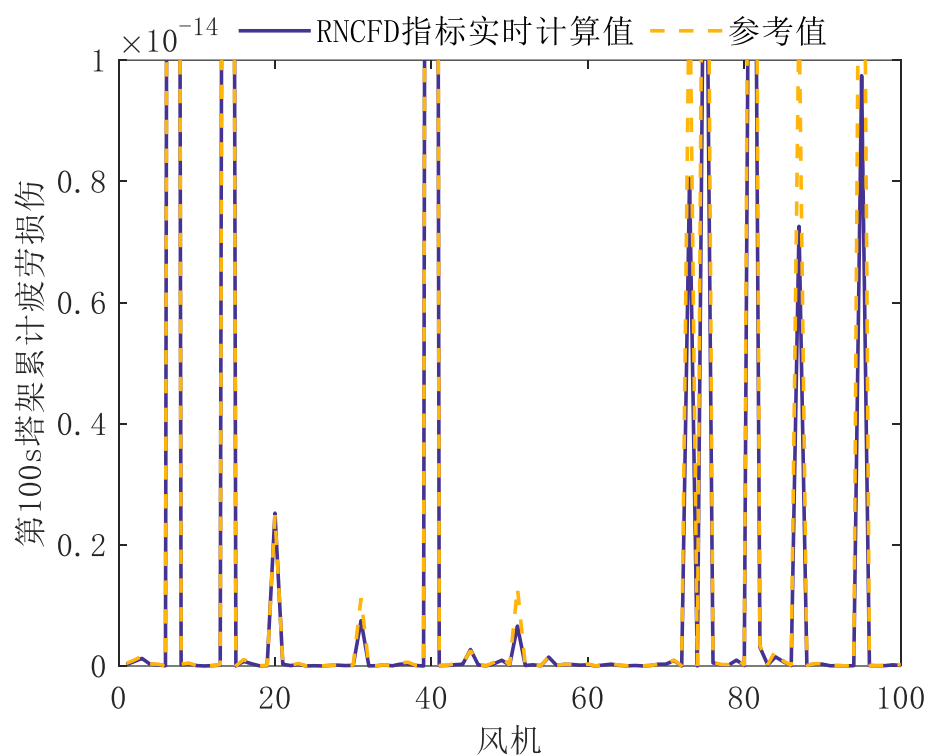
劳损伤最严重，7 号风机、14 号风机紧随其后，三者的累积疲劳损伤远超其余风机，并且 73 号风机、75 号风机、81 号风机、87 号风机、95 号风机的累积疲劳损也十分严重，这 8 台风机塔架的累积疲劳损伤超过其余风机多个数量级。

此外，通过对比 RNCFD 指标与附件 1 给定的参考值，可以发现，第 100s 的 RNCFD 指标与参考值误差在允许范围内，误差主要由二者指标计算方式不同导致，并且各个风机之间的 RNCFD 指标分布也与参考值的分布相同，说明实时从第 1s 计算到第 100s 的 RNCFD 指标可以准确地评估各风机元件的累积疲劳损伤分布。

图 4.3 主要从最大累积疲劳损伤的元件的损伤量级去观察其余元件，可能导致低损伤量级的元件无法观察清楚，下面将图 4.3(a)纵轴量级分别调到 10^{-13} 、 10^{-15} 、 10^{-17} ，图 4.3(b)纵轴量级调到 10^{-14} 、 10^{-15} 、 10^{-16} ，再次观察各风机元件第 100s 累积疲劳损伤分布，其结果如图 4.4(a)、(b)，4.5(a)、(b)，4.6(a)、(b)所示。

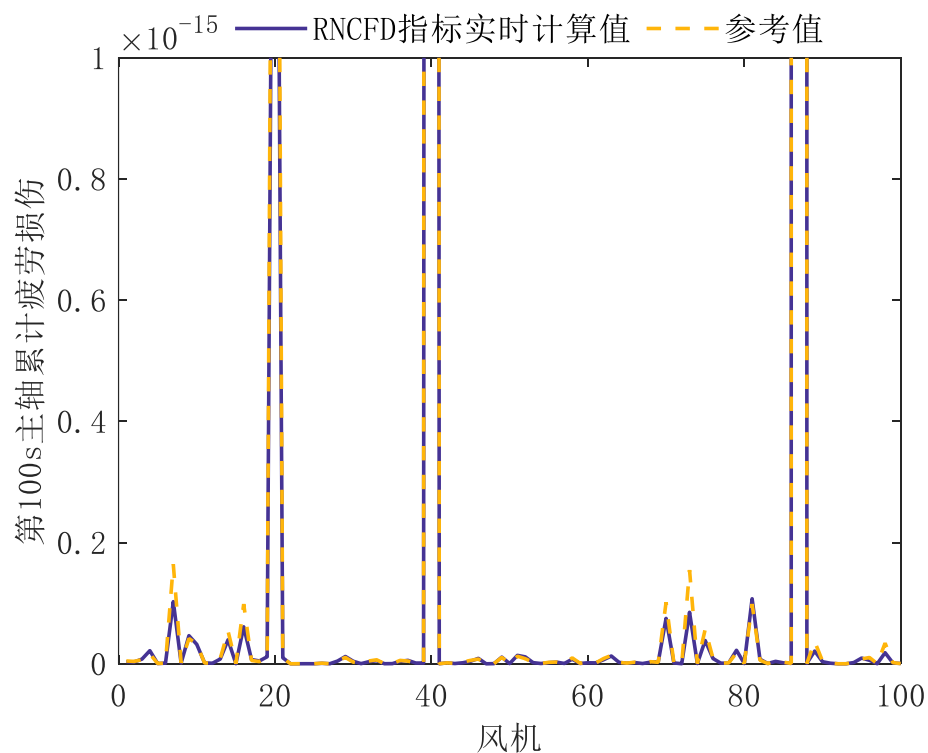


(a) 各风机主轴第 100s 累积疲劳损伤分布（纵轴量级 10^{-13} ）

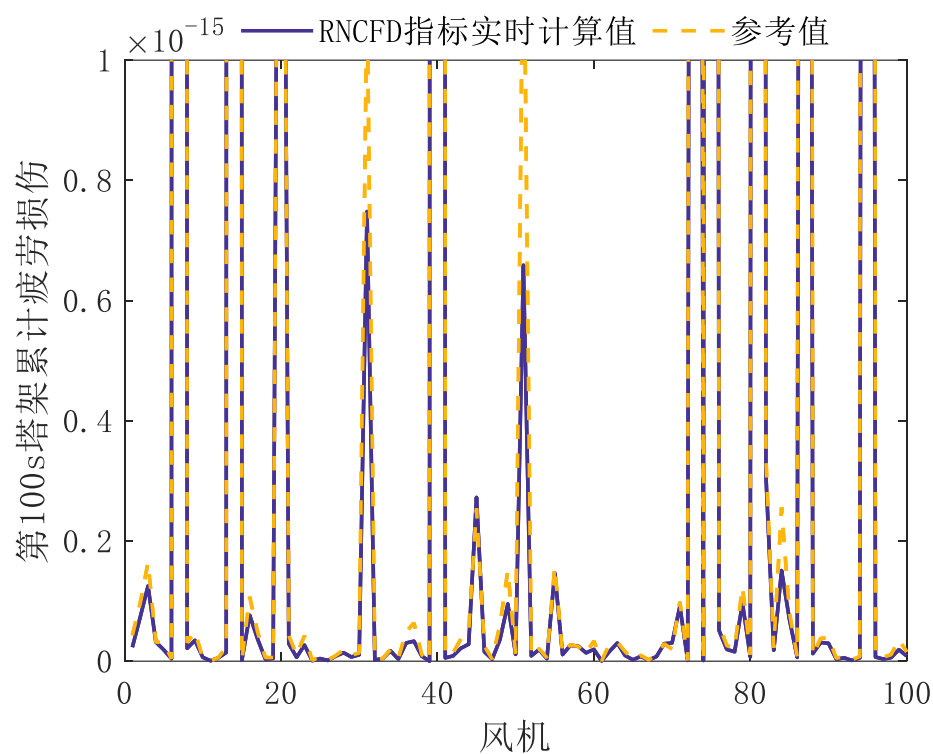


(b) 各风机塔架第 100s 累积疲劳损伤分布 (纵轴量级 10^{-14})

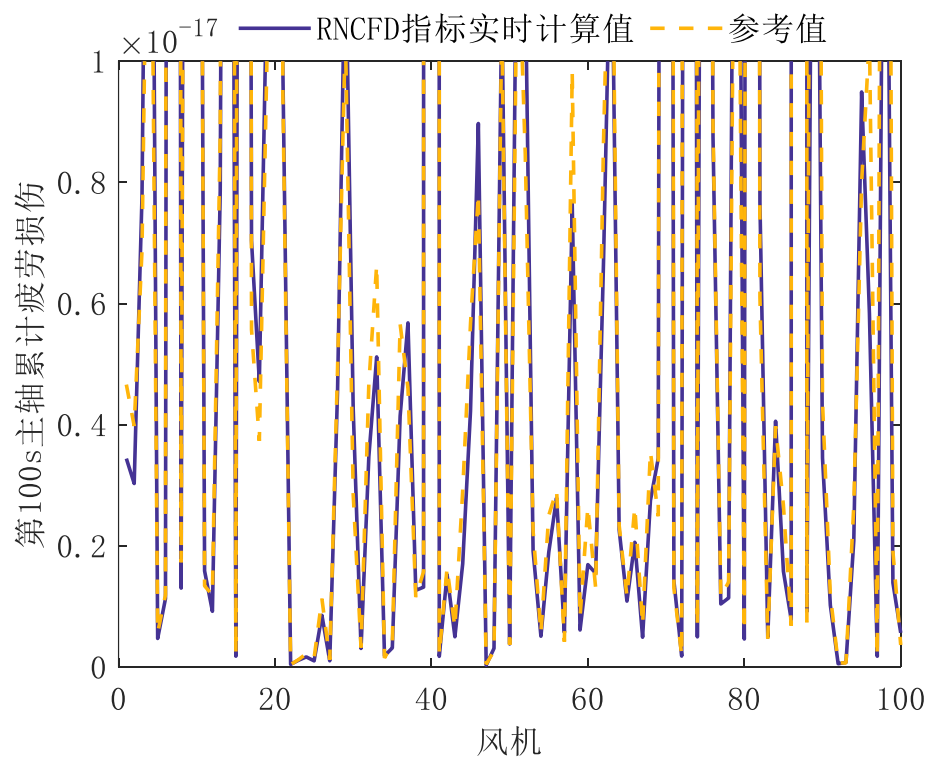
图 4.4 各风机元件第 100s 累积疲劳损伤分布 (不同纵轴量级 1)



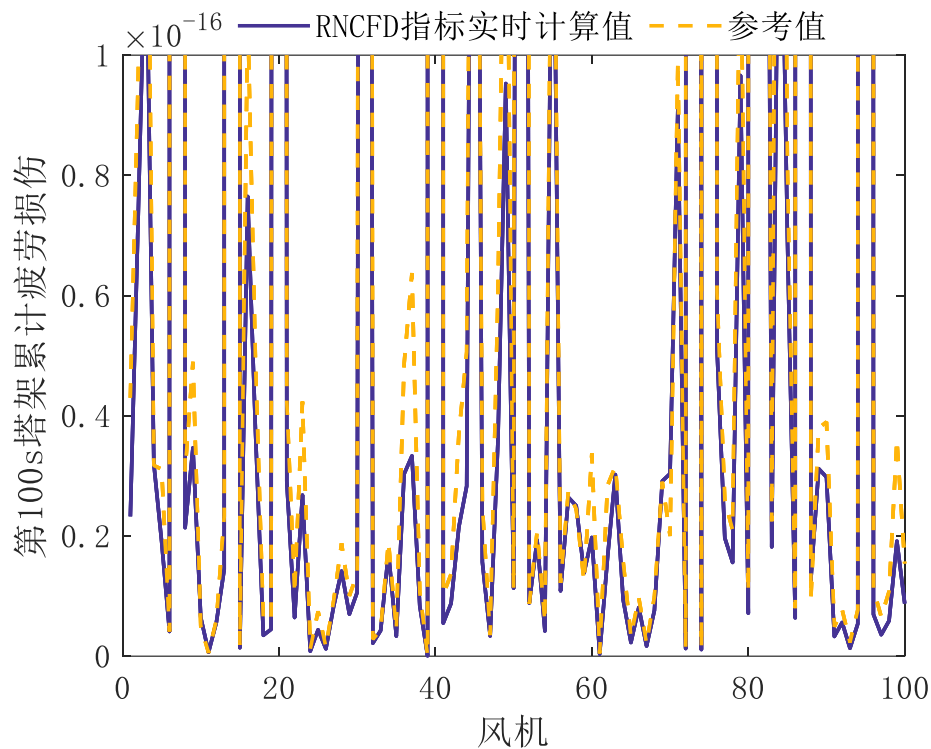
(a) 各风机主轴第 100s 累积疲劳损伤分布 (纵轴量级 10^{-15})



(b) 各风机塔架第 100s 累积疲劳损伤分布 (纵轴量级 10^{-15})
图 4.5 各风机元件第 100s 累积疲劳损伤分布 (不同纵轴量级 2)



(a) 各风机主轴第 100s 累积疲劳损伤分布 (纵轴量级 10^{-17})



(b) 各风机塔架第 100s 累积疲劳损伤分布（纵轴量级 10^{-16} ）

图 4.6 各风机元件第 100s 累积疲劳损伤分布（不同纵轴量级 3）

分析图 4.4，图 4.5，图 4.6 可知，各个风机之间的 RNCFD 指标分布与参考值的分布基本相同，说明实时从第 1s 计算到第 100s 的 RNCFD 指标可以比较准确地评估各风机元件的累积疲劳损伤分布。

上述分析验证了所提 RNCFD 指标的准确性，对于 RNCFD 指标的实时性，下面将通过分析各风机累积疲劳损伤实时增长情况进行验证。

4.3.3 各风机元件累积疲劳损伤实时增长情况

对于各风机累积疲劳损伤实时增长情况，由于风机数量过多，难以全部用图片展示，因此为表明典型代表性，需选择合适的代表风机的累积疲劳损伤实时增长情况图片来验证 RNCFD 指标的实时性。

首先说明代表性风机选择结果与选择原因，根据上小节各风机第 100s 的累积疲劳损伤分布情况选择累积疲劳损伤最多的风机，这些风机元件损伤严重是由于出现了极大载荷循环，需要能够实时地判断极大载荷循环出现的时间，通过分析这些风机的累积疲劳损伤实时增长情况可以更为有效地验证 RNCFD 指标的实时性，因此选择风机元件第 100s 的累积疲劳损伤最严重的几台风机作为代表风机。由上小节可知，主轴累积疲劳损伤最严重的为 87 号风机；塔架累积疲劳损伤比较严重的风机为 40 号风机、7 号风机、14 号风机、73 号风机、75 号风机、81 号风机、87 号风机、95 号风机。总计 8 台风机元件累积疲劳损伤实时增长情况，其结果如图 4.7、图 4.8 所示。

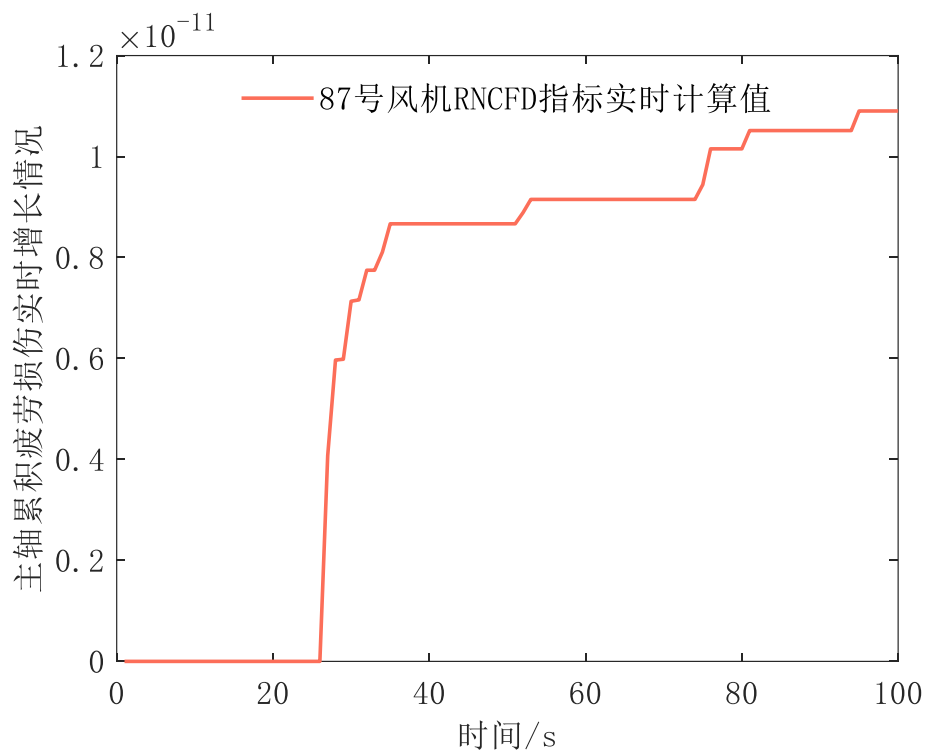


图 4.7 87 号风机主轴累积疲劳损伤 100s 实时增长情况

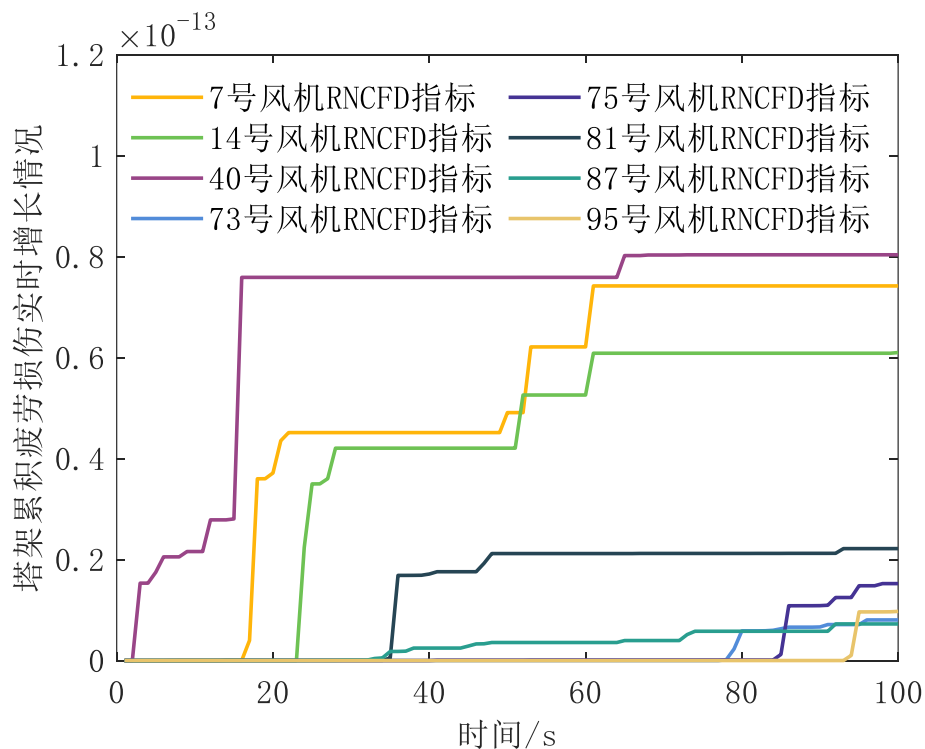


图 4.8 7、14、40、73、75、81、87、95 号风机塔架累积疲劳损伤 100s 实时增长情况

分析图 4.7 可知，87 号风机主轴在 27 秒出现载荷大扰动，RNCFD 指标快速增长，，说明 87 号风机在 27s 左右出现巨大扰动波及主轴，导致主轴载荷产生巨大变化。通过分析所给载荷数据，见图 4.9，验证了所提猜想。

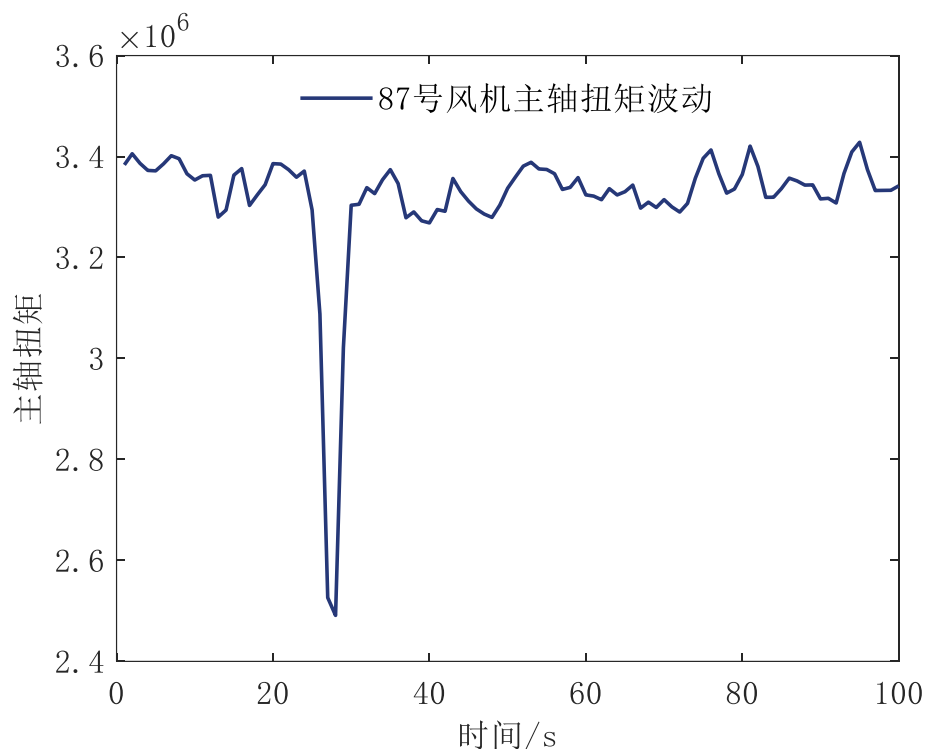


图 4.9 87 号风机主轴扭矩实时波动情况

对于其余风机的大扰动出现时间情况分析，可仿照上文思路进行分析，现说明分析结果。分析图 4.8 可知，7 号风机塔架第 18s 出现载荷大扰动；14 号风机塔架第 25s 出现载荷大扰动；40 号风机塔架第 1s 就出现载荷大扰动，到第 16s 才逐渐结束；73 号风机塔架第 80s 出现载荷大扰动；75 号风机第 86s 出现载荷大扰动；81 号风机塔架第 36s 出现载荷大扰动；95 号风机塔架第 95s 出现载荷大扰动，将分析结果与所给载荷数据进行比较，上述分析结果均正确，限于篇幅，具体 7、14、40、73、75、81、95 号风机的载荷实时波动情况图不进行展示验证。

通过上述典型代表风机元件的累积疲劳损伤实时增长情况，验证了所提 RNCFD 指标的实时性，能够快速反应关键时刻的载荷波动。

五、问题二：模型建立与求解

5.1 风力发电机组部件

风力发电机组包括风力机、齿轮机、双馈异步发电机、网侧变频器、机侧变频器等重要部件，如图 5.1 所示：

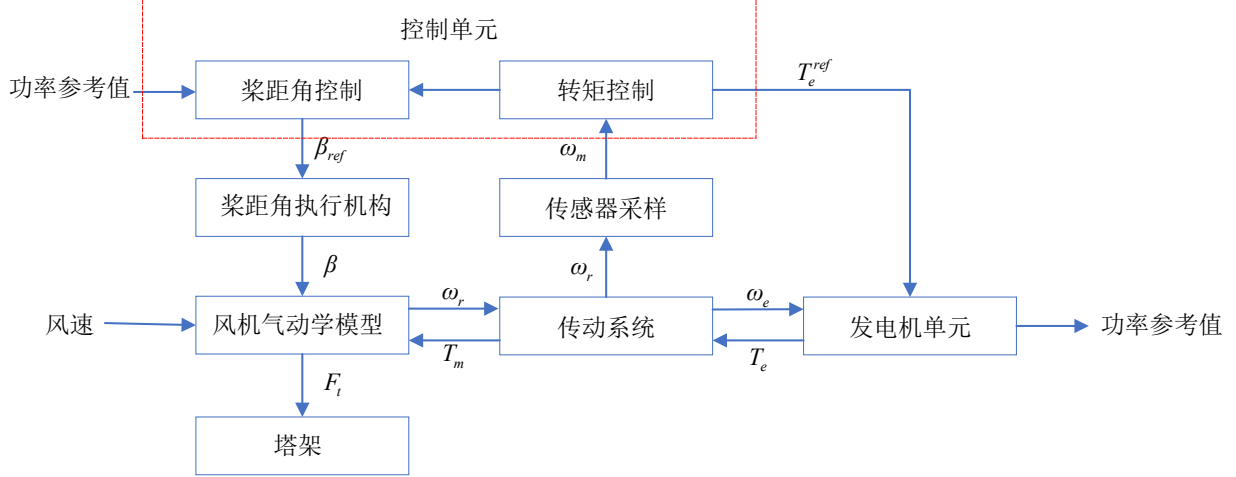


图 5.1 风力发电机组部件

发电过程如下：风力推动风轮旋转，风轮将风能转化为机械能，风轮通过主轴与发电机相连，将机械能传递给发电机转子，定子通过电磁感应现象发电，将机械能转化为电能，定子直接与电网相连，通过变频器将电能输送出去。

风机响应电网调度的过程为：电网向风电站发出调度指令，风电站集中计算每台风机应发功率，即风机功率参考值；风机根据指令、当前风速进行控制，包括桨距角控制、与风速相关的转矩控制，这是强非线性过程，相关公式在之后介绍，由于风机难以实现一次动作便达到功率参考值，因而需要进行多次反馈、修正才能到达功率参考值附近，因此需要一定的时间动作。通过对附件 2 中输入功率参考值和实际输出功率的对比，发现二者存在较强的时延性，主要原因在于风机控制、调整过程不能实时完成，例如 t_0 时刻下达指令， t_1 时刻动作完成，从而导致 t_1 时刻的输出功率实际是参考 t_0 时刻指令控制的。

5.2 风力发电机组模型

气流动能公式为：

$$E = \frac{1}{2}mv^2 \quad (5.1)$$

式中： m 为气体质量（ kg ）， v 为气体流速，即风速（ m/s ）。

进一步改写，得到风能为：

$$E = \frac{1}{2}\rho Vv^2 = \frac{1}{2}\rho Av^3 \quad (5.2)$$

式中： ρ 为空气密度（ kg/m^3 ）， V 为单位时间内流过截面积 A 的体积，有 $V = Av$ 。

可以看出，风能与风速的三次方成正比。风能无法直接转化为风轮的机械能，存在下列转化公式：

$$P_m = 0.5\rho\pi R^2 v^3 C_p \quad (5.3)$$

式中： P_m 为风轮机械能（ J ）， R 为风机风轮半径（ m ）， C_p 为风能利用系数，表征风能转化为机械能的效率。

风能利用系数 C_p 由下列公式确定：

（1）低风速下：

$$C_p = (0.44 - 0.0167\beta) \sin\left[\frac{\pi(\lambda - 3)}{15 - 0.3\beta}\right] - 0.00184(\lambda - 3)\beta$$

$$\lambda = \frac{\omega_r R}{v}$$
(5.4)

式中： λ 为叶尖速比， β 为桨距角， ω_r 为风轮转速，即低速轴转速。

（2）高风速下：

$$C_p = 0.5176(116\frac{1}{\lambda_1} - 0.4\beta - 5)e^{-\frac{21}{\lambda_1}} + 0.0068\lambda$$

$$\frac{1}{\lambda_1} = \frac{1}{\lambda + 0.08\beta} - \frac{0.035}{\beta^3 + 1}$$

$$\lambda = \frac{\omega_r R}{v}$$
(5.5)

从上述两式可以看出，风能利用系数 C_p 与叶尖速比 λ 、桨距角 β 相关。

风机输出功率 P_o 与机械功率之间关系如下：

$$P_o = \eta P_m$$
(5.6)

式中： η 为风机机械功率转化为输出功率的效率。

得到风轮机械功率后，可以计算风机主轴扭矩：

$$T_m = \frac{0.5\pi\rho R^2 v^3 C_p}{\omega_r} = \frac{P_m}{\omega_r}$$
(5.7)

对于塔架推力，引入为风能利用系数 C_t ，计算公式为：

$$F_t = 0.5\pi\rho R^2 v^2 C_t(\lambda, \beta)$$
(5.8)

C_t 同样为与叶尖速比 λ 、桨距角 β 相关，且为强非线性关系，难以量化，因此本文引入轴向诱导因子 $a(0 \leq a \leq 1)$ ， a 表征风速通过风机的流速衰减，计算公式为：

$$a = \frac{v - v_d}{v}$$
(5.9)

式中： v_d 为风速衰减后的流速。

对于风能利用系数 C_p 、风能利用系数 C_t ，也可使用轴向诱导因子 a 进行描述，公式分别为：

$$C_p = 4a(1-a)^2$$
(5.10)

$$C_t = 4a(1-a)$$
(5.11)

进一步可以得到：

$$P_m = \frac{1}{2} \pi \rho R^2 v^3 \cdot 4a(1-a)^2$$
(5.12)

$$F_t = 0.5\pi\rho R^2 v^2 \cdot 4a(1-a) = \frac{P_m}{v(1-a)}$$
(5.13)

从而在给定风速下可以求得轴向诱导因子 a ，进一步求得塔架推力。

5.3 风机塔架推力和主轴扭矩估计模型

5.3.1 风机输出功率估计模型

5.2 节对风机各变量之间的关系进行了梳理，其输出功率是重要的中间变量，因此首先对其进行估算。

5.1 节已经提到，风机通过电网下达的指令进行自身控制调整不能实时完成，其输出功率与功率参考值存在关系，以所给数据风机 1 为例，画出当前时刻风机 1 输出功率 P_o^t 与上一时刻参考功率 P_{ref}^{t-1} 的散点图如图 5.2 所示：

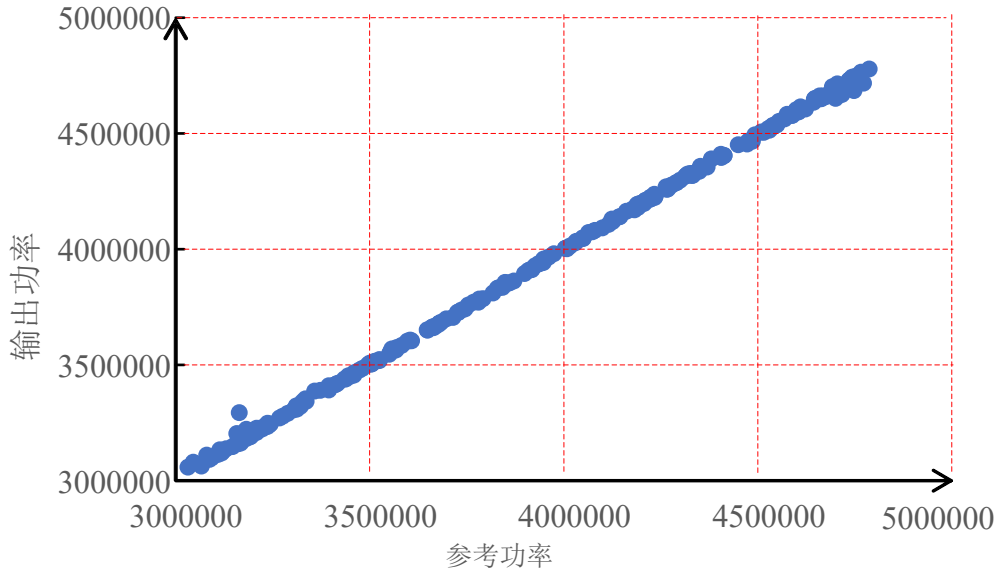


图 5.2 风力发电机组部件

从上图可以看出，风机输出功率 P_o^t 与上一时刻参考功率 P_{ref}^{t-1} 存在强线性关系，二者关系图基本分布在斜率为 1 的直线附近，即存在如下关系：

$$P_o^t = k_1 P_{ref}^{t-1} \quad (5.14)$$

也可以看出，二者之间存在一定的误差，5.1 节中发分析了风机在风电站指令下的动作过程，其输出功率与当前时刻风速存在关系；一般来说，风速与功率的关系为 3 次方关系，如式 2 所示，因此使用最小二乘法拟合、制订的拟合公式如下：

$$P_o^t = k_1 P_{ref}^{t-1} + k_2 v_t^3 + k_3 \quad (5.15)$$

式中： v_t 为当前时刻风速， k_3 为进行修正的偏置项。

5.3.2 风机主轴扭矩估计模型

根据公式 (5.6) 和公式 (5.7)，在 5.3.1 小节的基础上可以得到主轴扭矩。

5.3.3 风机塔架推力估计模型

根据公式 (5.12) 和公式 (5.13)，在 5.3.1 小节的基础上可以得到主轴扭矩。

5.4 求解结果与分析

5.4.1 风机输出功率估计模型求解结果展示

采用误差平方和指标、平均绝对百分比误差表征估计结果与实际结果的差距，公式分别为：

$$e_{SE} = \sum_{i=1}^n (x_{i_p} - x_i)^2 \quad (5.16)$$

$$e_{MAPE} = \frac{1}{n} \times \frac{|x_{i-p} - x_i|}{x_i} \times 100\% \quad (5.17)$$

式中： x_i 为实际值， x_{i-p} 为估计值， n 为统计的时刻总数。

风场 1、风场 2 中 100 个时刻下，100 台风机输出功率误差平方和如图 5.3、5.4 所示：

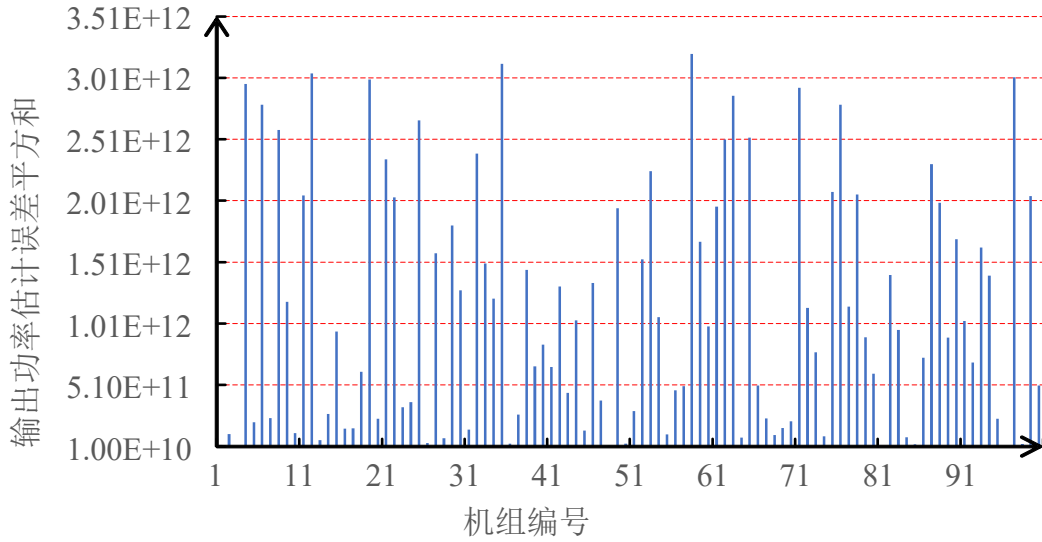


图 5.3 风场 1 风机输出功率估计误差平方和

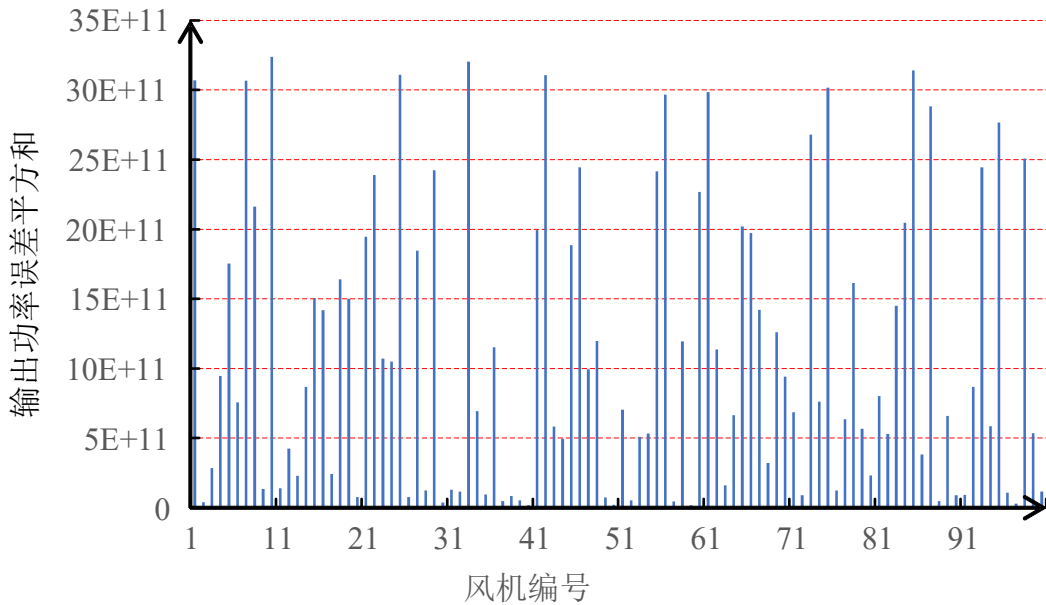


图 5.4 风场 2 风机输出功率估计误差平方和

从上述两幅图可以看出，两个风场 100 台风机输出功率估计误差平方和基本位于 2.5×10^{12} 以下，为与真实输出功率进行对比，图 5.5、图 5.6 画出来风场 1、风场 2 中 100 台风机输出功率平均绝对误差百分比。

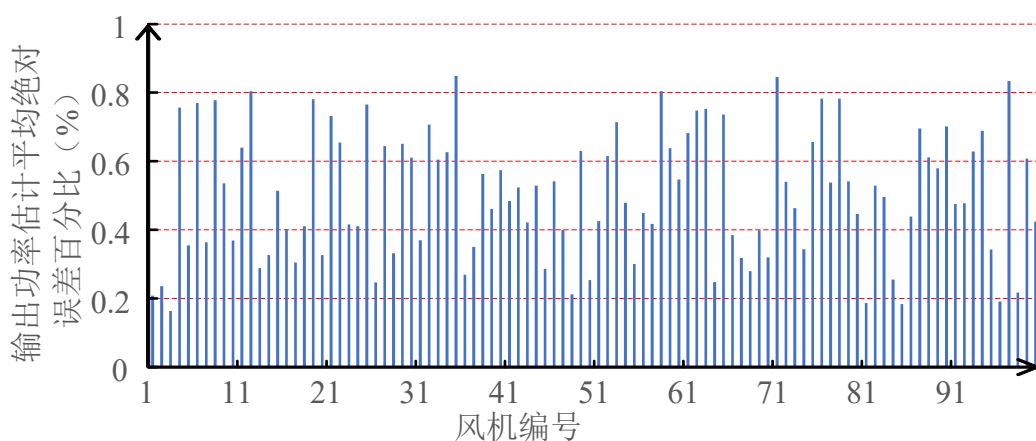


图 5.5 风场 1 风机输出功率估计平均绝对误差百分比

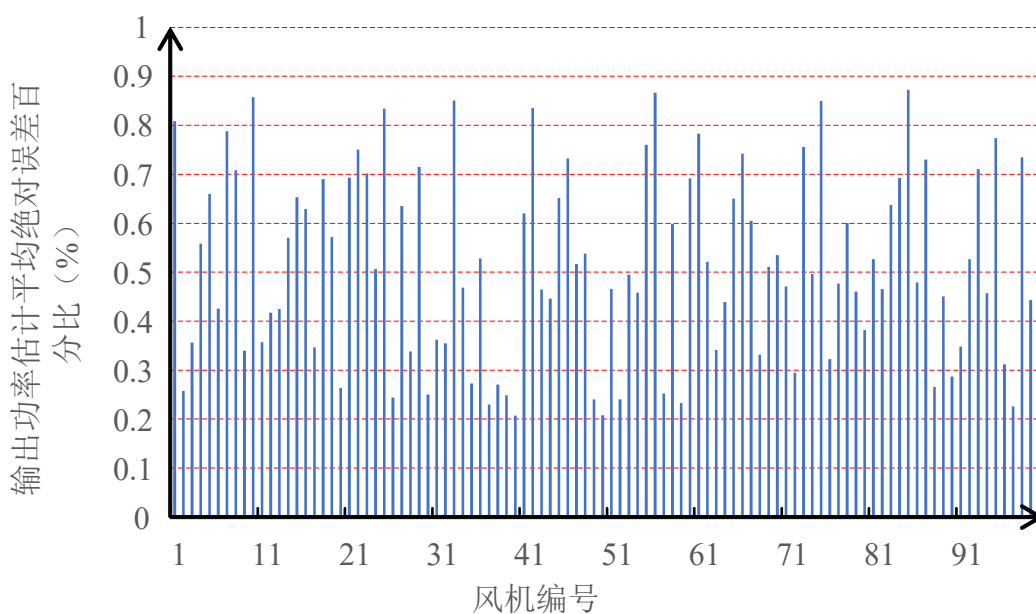


图 5.6 风场 2 风机输出功率估计平均绝对误差百分比

从上述两幅图可以明显的看出所估计的输出功率与真实输出功率的误差不超过 1%，说明了所提估计方法的准确性。

5.4.2 风机主轴扭矩估计模型求解结果展示

100 个时刻下，风场 1、风场 2 中 100 台风机主轴扭矩估计误差平方和如图 5.7、5.8 所示：

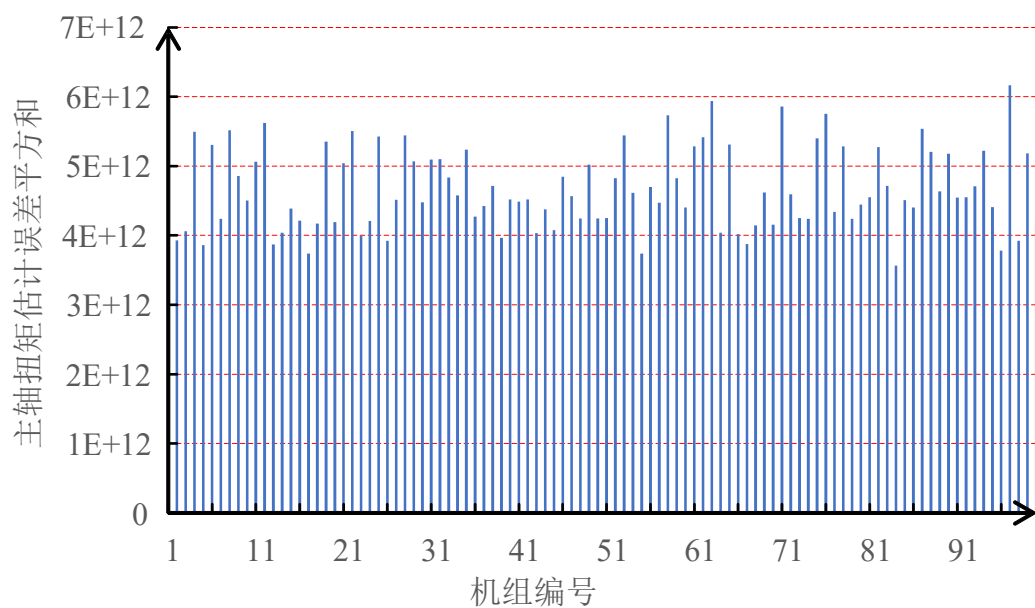


图 5.7 风场 1 风机主轴扭矩估计误差平方和

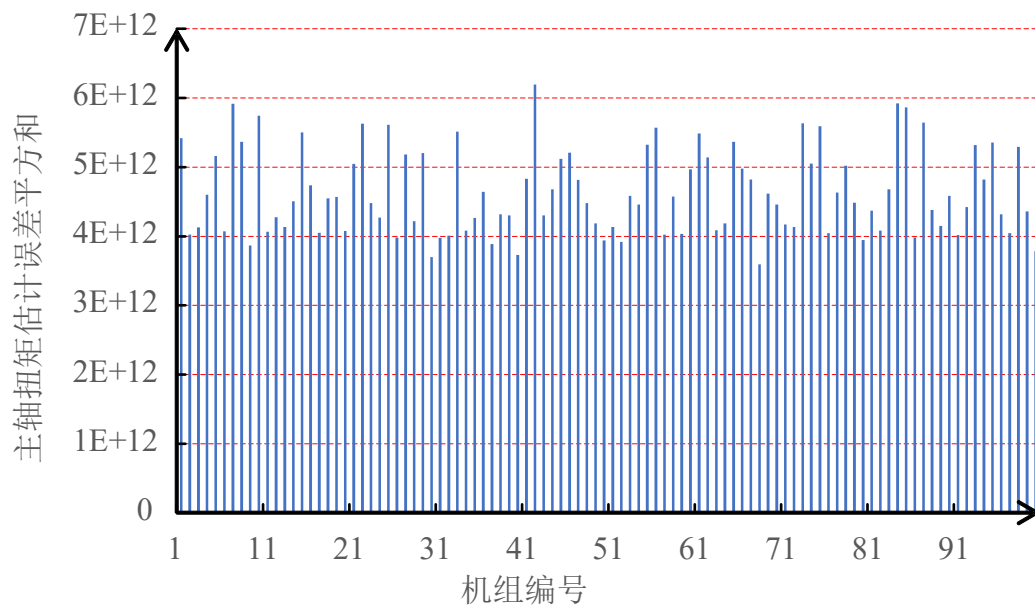


图 5.8 风场 2 风机主轴扭矩估计误差平方和

从上述两幅图可以看出，两个风场 100 台风机输出功率估计误差平方和基本位于 5×10^{12} 以下，为与真实输出功率进行对比，图 5.9、图 5.10 画出来风场 1、风场 2 中 100 台风机主轴扭矩估计的平均绝对误差百分比。

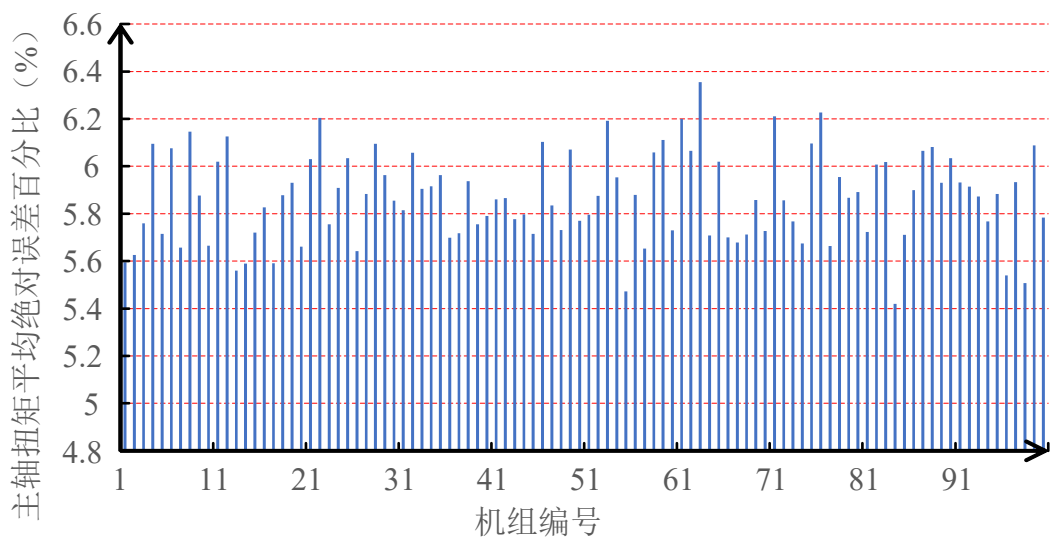


图 5.9 风场 1 风机主轴扭矩估计平均绝对误差百分比

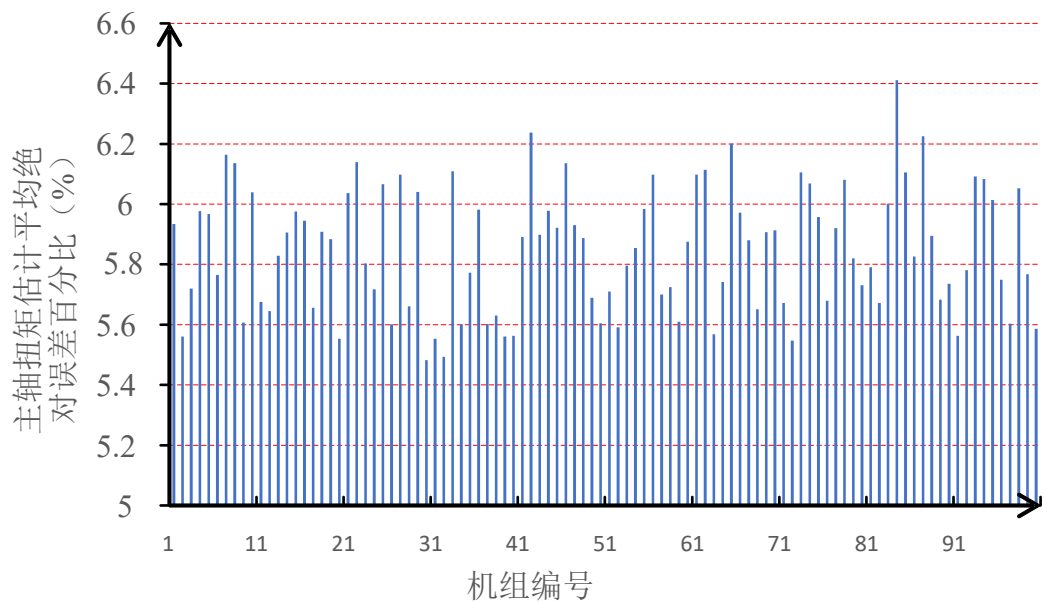


图 5.10 风场 2 风机主轴扭矩估计平均绝对误差百分比

从上述两幅图可以明显的看出所估计的主轴扭矩与真实主轴扭矩的误差不超过 6.6%，说明了所提估计方法的准确性。

5.4.3 风机塔架推力估计模型求解结果展示

风场 1、风场 2 中 100 台风机在 100 个时刻下的塔架推力估计的误差平方和如图 5.11、图 5.12 所示：

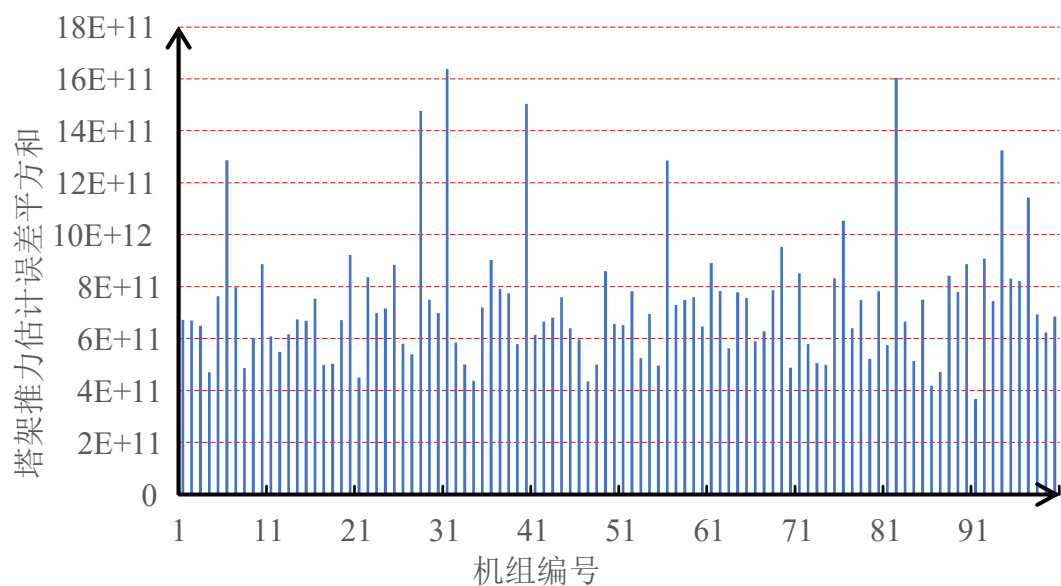


图 5.11 风场 1 风机塔架推力估计误差平方和

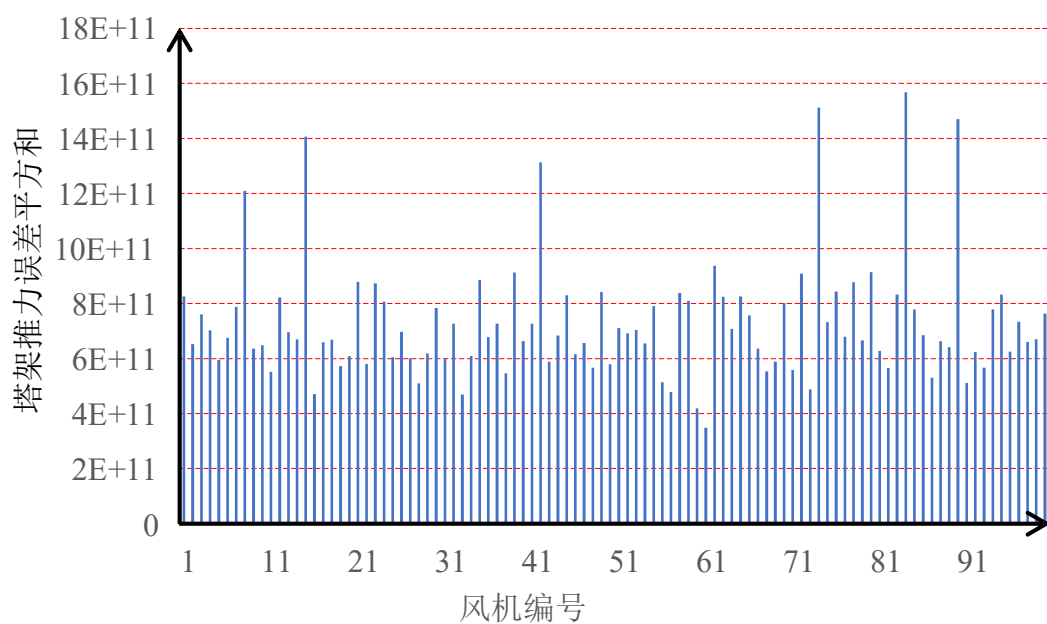


图 5.12 风场 2 风机塔架推力估计误差平方和

从上述两幅图可以看出，两个风场 100 台风机输出功率估计误差平方和基本位于 5×10^{12} 以下，为与真实输出功率进行对比，图 5.13、图 5.14 画出来风场 1、风场 2 中 100 台风机塔架推力估计的平均绝对误差百分比。

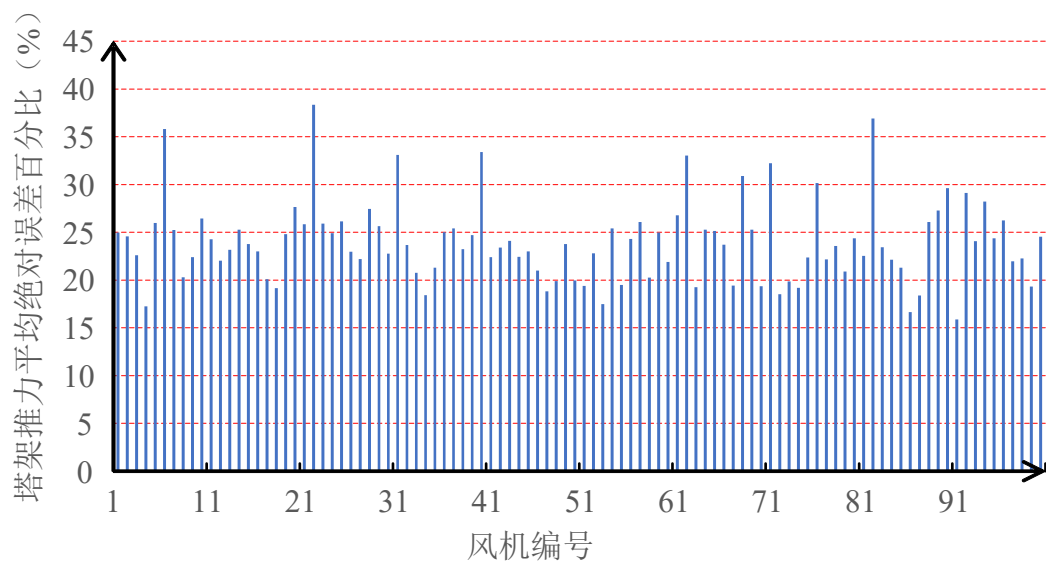


图 5.13 风场 1 风机塔架推力估计平均绝对误差百分比

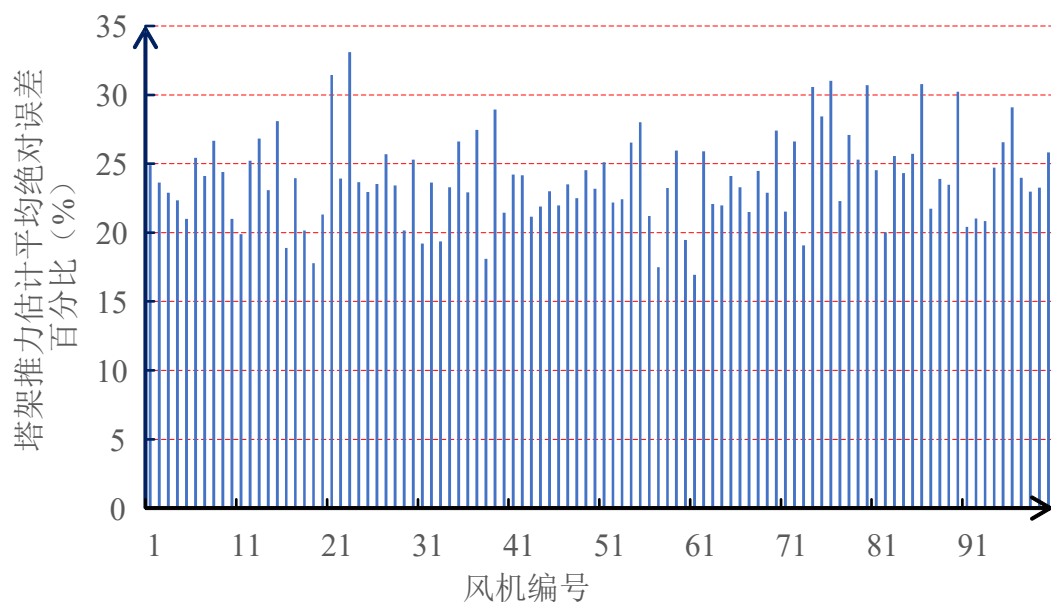


图 5.14 风场 2 风机塔架推力估计平均绝对误差百分比

从上述两幅图可以明显的看出所估计的塔架扭矩与真实输出功率的误差大概位于 20% 左右，处于可以接受的范围。

六、问题三：模型建立与求解

6.1 基于强化学习的风电场功率分配模型

6.1.1 基于强化学习的风电场功率分配建模

本文采用风机累积疲劳损伤描述风机的疲劳程度，因此目标函数如下：

$$\min \sum_{t=1}^{T_{all}} \sum_{i=1}^{N_w} (\varepsilon_1 \cdot D_{t,i}^T + (1-\varepsilon_1) \cdot D_{t,i}^F) \quad (6.1)$$

式中： $D_{t,i}^T$ 为归一化后主轴累积疲劳损伤； $D_{t,i}^F$ 为归一化后塔架累积疲劳损伤，其具体表达式见问题 1 提出的实时非线性累积疲劳损伤指标；归一化的原因在于 $D_{t,i}^T$ 与 $D_{t,i}^F$ 是两个不同的类型量，实际值相差几个数量级，进行多目标优化的时候影响较大，进行归一化操作以便优化，放缩 4 个量级； T_{all} 为优化的时段，本文中为 100 个时刻，之所以使用总的优化时段，是因为如果只对当前时刻进行优化，只关注了当前的信息，而忽略了过去、未来的信息，容易陷入局部最优解，因此直接从全局进行优化，保证总的时段累积疲劳损伤最小； N_w 为风机总数量，本文中为 100 台风机； ε_1 为目标权重，两个目标权重和为 1，由于进行了归一化，本文直接取 0.5。

约束条件包括功率平衡约束：

$$\sum_{i=1}^n P_{wt,i}^{ref} = P_{wf}^{ref} \quad (6.2)$$

式中： P_{wf}^{ref} 为调度下给风电场的功率值， $P_{wt,i}^{ref}$ 为各台风机的分配功率。

风机功率上下限约束：

$$P_{wt,i}^{\min} \leq P_{wt,i}^{ref} \leq P_{wt,i}^{\max} \quad (6.3)$$

式中： $P_{wt,i}^{\min}$ 、 $P_{wt,i}^{\max}$ 分别为当前风速下的最小最大值。

风机功率偏移约束：

$$|P_{wt,i}^{ref} - P_{wt,i}^{av}| \leq 1 \quad (6.4)$$

式中： $P_{wt,i}^{av}$ 为平均功率分配下的指标，优化的功率 $P_{wt,i}^{ref}$ 与其偏移不超过 1MW。

基于如上的建模公式，本文搭建基于强化学习的智能功率分配代理模型，并采用动作、状态、奖励三元组对代理进行运算，从而对动作、状态、奖励进行针对性设置、选择。模型如下图所示，三元组各含义如下：

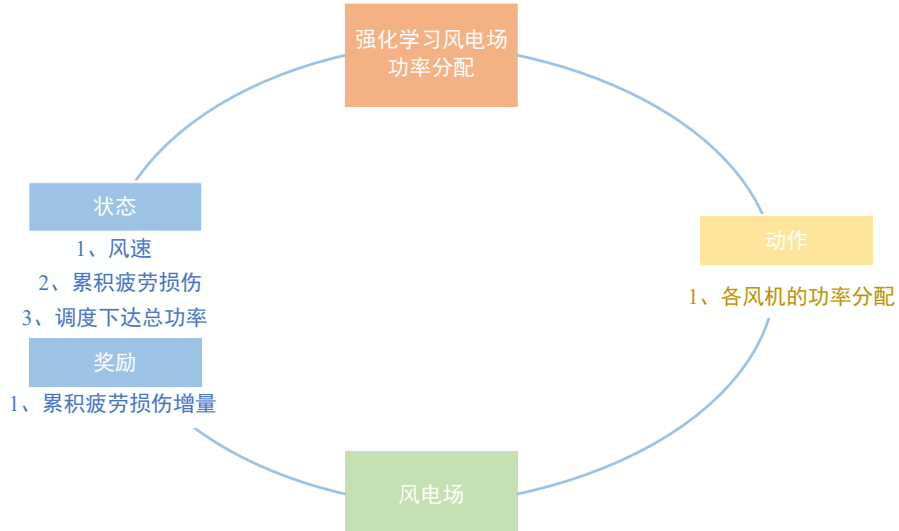


图 6.1 风电场功率分配代理结构

(1) 动作:

风电场功率分配的主要为各风机的功率分配，公式如下:

$$A = \{P_{wt,i}^{ref}\} \quad (6.5)$$

式中: A 为代理的动作空间, 且各风机分配功率 $P_{wt,i}^{ref}$ 为连续动作变量。

(2) 状态:

实时风电场状态是代理的主要输入, 包括调度总功率、各风机当前时刻风速、累积疲劳损伤, 因此状态空间的定义如下:

$$S = [P_{wf}^{ref}, D_{i,总}, v_i], i \in [1, N_w] \quad (6.6)$$

式中: P_{wf}^{ref} 为调度总功率, $D_{i,总}$ 为归一化之后每台风机的总累积疲劳损伤, v_i 为各风机风速, N_w 为风电场风机总数。

(3) 奖励:

风电场功率分配代理基于奖励更高的动作进行决策, 奖励设计将直接影响到强化学习算法的优化方向, 同时更为直接有效的奖励设计也有助于加快算法的训练速度, 提升算法的优化效果。因此, 功率分配的奖励函数关注于尽可能减少总累积疲劳损伤, 由于代理向着奖励增大的方向动作, 同时因为总累积疲劳损伤必然是逐渐增大的, 我们的目标是尽可能使其减小, 故使用总累积疲劳损伤的增加值, 其增加值越小说明当前的累计疲劳损伤最小, 故而使用总累积疲劳损伤增加值的相反数作为奖励, 定义奖励的公式为:

$$R_{cost} = 1 - \Delta D_{i,总} \quad (6.7)$$

式中: R_{cost} 为奖励值, $\Delta D_{i,总}$ 是每台风机的总疲劳损伤值; 当风机功率平衡不满足安全约束时, 额外引入较大的负值奖励作为惩罚, 因此总奖励计算为:

$$R = R_{cost} + R_{pen} \quad (6.8)$$

式中: R_{pen} 为负值奖励, R 为代理总奖励。

6.1.2 深度确定性策略梯度

深度确定性策略梯度 (Deep deterministic policy gradient, DDPG) 是一种经典的连续动作强化学习算法, 其基于确定性策略梯度和行动者-批评家方法开发。确定性策略梯度旨在根据策略值函数的梯度上升方向来训练代理模型。该方法定义为:

$$\nabla_{\theta} J(\pi_{\theta}) = E_{s \sim \rho^{\pi}} [\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q_{\pi}(s, a) |_{a=\pi_{\theta}(s)}] \quad (6.9)$$

式中: θ 是代理中的可训练参数集, J 是累积奖励, π_{θ} 是策略函数, ρ^{π} 是状态的采样空间。DDPG 在确定性策略梯度法的基础上, 建立了选择确定性策略的演员模型和评估行动值函数的评论家模型。此外, 演员和评论家在 DDPG 中都有两个副本, 即主模型和目标模型。这两个模型副本可以异步更新它们的可训练参数, 从而预防强化学习算法中价值高估的问题, 使算法顺利收敛并取得更高的优化效果, 具体的模型结构如下图所示:

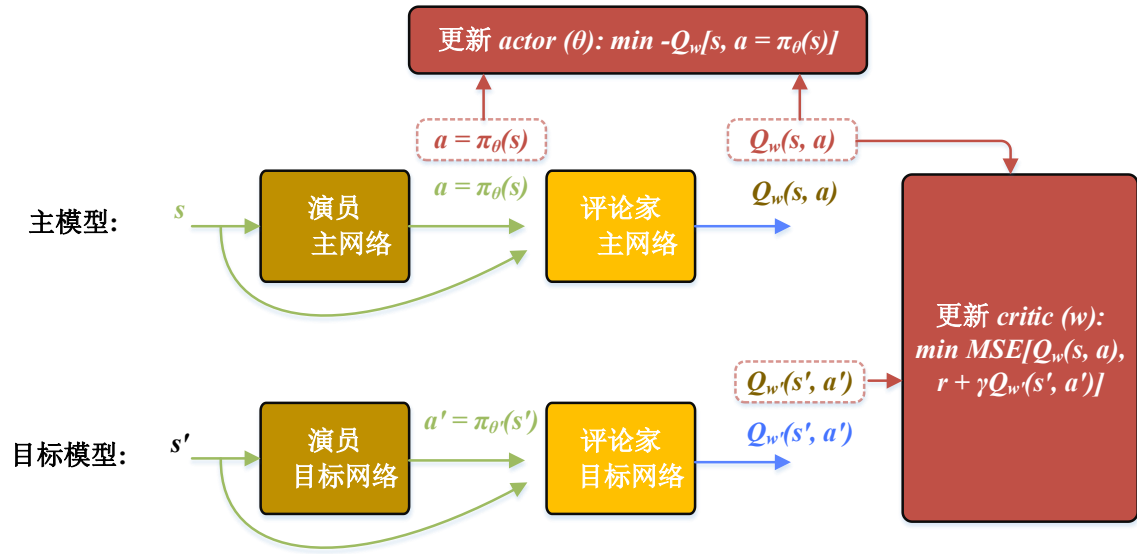


图 6.2 DDPG 模型结构

首先，演员网络以风电场下一个初始运行时刻的状态为输入、以有源配电网下一个运行时刻的功率分配动作作为输出，公式表示为：

$$A_{t+1} = M_A(X_{t+1}) \quad (6.10)$$

式中： M_A 为演员网络， X_{t+1} 为有源配电网下一个运行时刻的状态， A_{t+1} 为有源配电网下一个运行时刻的转供动作。

其次，构建DDPG模型的价值评估网络即评论家网络，其以风电场下一个运行时刻的状态和功率分配动作为输入、以风电场下一个运行时刻的功率分配奖励值的评估结果为输出。评论家网络并不会直接影响决策，但其通过不断学习来拟合状态价值函数，为演员网络决策的精确评判和参数更新打下基础，从而间接地引导演员网络的决策方式。公式表示为：

$$Q_{t+1} = M_C(X_{t+1}, A_{t+1}) \quad (6.11)$$

式中： M_C 为评论家网络， X_{t+1} 为风电场下一个运行时刻的状态， A_{t+1} 为风电场下一个运行时刻的功率分配动作， Q_{t+1} 为风电场下一个运行时刻的功率分配的评估结果；

然后，对DDPG模型的演员网络进行训练，训练目标函数的公式表示为：

$$\max Q_{t+1} [X_{t+1}, A_{t+1} = M_A(X_{t+1})] \quad (6.12)$$

式中：目标函数在于最大化风电场下一个运行时刻的功率分配奖励值的评估结果 Q_{t+1} ， X_{t+1} 为风电场下一个运行时刻的状态， A_{t+1} 为风电场下一个运行时刻的功率分配动作， M_A 为动作表演网络。

最后，对DDPG模型的评论家网络进行训练，训练目标函数的公式表示为：

$$\min \left\| Q_{t+1}(X_{t+1}, A_{t+1}) - \left(R_{t+1} + \gamma M_c^{copy} \left(X_{t+1}, M_A^{copy}(X_{t+1}) \right) \right) \right\|_2^2 \quad (6.13)$$

式中：目标函数在于最小化功率分配动作奖励值的评估误差， γ 为风电场运行时刻折扣系数， M_c^{copy} 为价值评估网络模型副本， M_A^{copy} 为动作表演网络模型副本， X_{t+1} 为风电场下一个运行时刻的状态， A_{t+1} 为风电场下一个运行时刻的功率分配动作， R_{t+1} 为风电场下一个运行时刻的功率分配动作奖励值， Q_{t+1} 为风电场下一个运行时刻的功率分配动作奖励值的评估结果。

6.1.3 功率分配安全约束匹配

保证约束条件是强化学习方法的难点，本小节对不满足约束条件的功率分配结果补充安全约束、进行修正。功率分配安全约束匹配技术的原理在于，功率分配等式约束在强化学习过程中已经满足，但功率分配结果可能不满足功率不等式约束，即式（2）和式（3），因此需要分配结果进行再分配，分配的步骤如下：

首先，对于越限风机 i ，将其分配功率 $P_{wt,i}^{ref}$ 限制为允许范围的边界值，例如对于风机 i ，通过平均分配法计算得到其功率分配为3MW，那么其优化后的分配功率满足： $P_{wt,i}^{ref} \in [2.5, 3.5](\text{MW})$ ，假设得到的 $P_{wt,i}^{ref}$ 越限为4MW，则先将其限制为3.5MW，显然会造成功率的不平衡，必须予以纠正；

然后计算所有机组限幅导致的功率不平衡总量和有功率裕度机组的功率裕度，为了避免分配过程造成其它机组的功率越限，通过下式将功率不平衡总量分配给有裕度的机组：

$$\Delta P_i = \frac{\Delta P}{\alpha_i} \quad (6.14)$$

式中： ΔP_i 为有功率裕度机组 i 的功率再分配量， α_i 为其分配系数，通过功率裕度进行描述，使功率裕度较大的机组承担的功率较大计算公式如下：

$$\alpha_i = \frac{P_i^{\lim}}{\sum_{i=1}^{N_{wt}} P_i^{\lim}} \quad (6.15)$$

式中： $\sum_{i=1}^{N_{wt}} P_i^{\lim}$ 为所有有功率裕度机组的裕度总和， $P_i^{\lim}$ 为有功率裕度机组 i 的功率裕度。

6.2 模型求解

在以上基础上，建立了如下图所示的基于强化学习的功率分配方法求解框架：

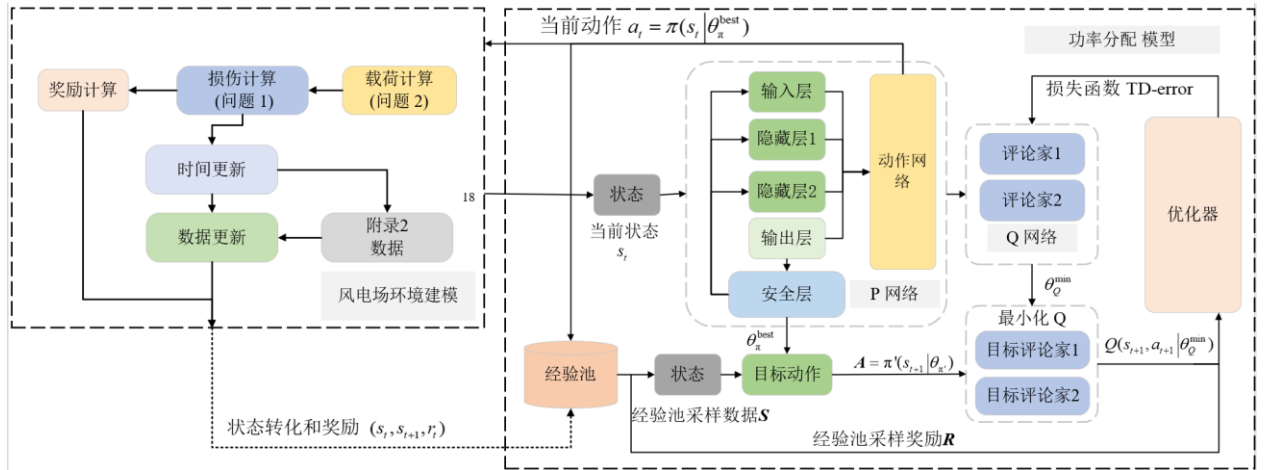


图 6.3 基于强化学习的功率分配方法

主要步骤包括：首先根据当前时刻初始化环境状态，输入到强化学习模型中，模型输出 100 台风机的调度决策传递至环境，环境通过内部计算获取设计的奖励值用以指导模型的训练迭代。细节如下：

(1)风电场环境建模：风电场环境接受模型给出的调度策略，并生成当前时刻的奖励指标。由于调度指令依靠神经网络获得，为了提升网络参数的拟合效率，建模中通常使用归一化技巧保证输入与输出限制在额定区间内。故第一步，环境将调取反归一化参数获取调

度策略真实值；随后，通过问题二所拟合的公式去分别计算每台风机的主轴扭矩和塔架推力，并输入到第一问设计的实时雨流法程序中。

这里累积损伤的计算程序使用 Matlab 平台进行编写，通过 matlabengine 在 python 中调用，实现两平台的动态交互。累计疲劳损伤的计算过程中，程序将实时保存和更新策略模型给出的功率分配数据，并用于下一次的损伤数据计算，因此本文的环境计算是全局且具有记忆性的，充分考虑了载荷的波形特性和时序关联。考虑到任务为 100s 调度，因此环境在第 100 次交互后重置，对应实时雨流法的程序也将同步初始化。

(2)神经网络建模调整：为了保证优化的高效性和准确性，本文设计了两类网络层级使得模型的输出策略尽可能的限制在可行域中：

对于功率平衡约束，本文仿照累加取反并放缩的方式避免不同机组的输出功率偏差过大，公式如下：

$$P_{out, scale, k} = \frac{(\text{Sigmoid}(P_{out, k}) * 0.5 + 0.625)}{\sum (\text{Sigmoid}(P_{out, i}) * 0.5 + 0.625)} \quad (6.16)$$

式中： $P_{out, scale, k}$ 为放缩后的神经网络输出表示当前机组调度功率占总功率的比值， $P_{out, k}$ 为神经网络输出，通过 Sigmoid 将数值限制在 0 到 1 之间。

经过放缩后的输出累加和为 1 天然满足功率约束，但可能不满足均值偏差 1MW 的功率约束。考虑到每台风机所有时刻均值约为 3.5MW，若此时设置其输出的比例系数为 0.01，对应上下限为 4.5MW 与 2.5MW，对应比例上下限为(0.007,0.013)。使用截断函数进行限幅：

$$P_{out, scale, k} = \begin{cases} 0.007 & P_{out, scale, k} < 0.007 \\ 0.013 & P_{out, scale, k} > 0.013 \\ P_{out, scale, k} & \text{else} \end{cases} \quad (6.17)$$

经过网络调整后，两个约束同时满足的概率超过 99%。对于极个别不满约束的情况通过上文所述的安全层进行功率的二次调整和分配。

6.3 求解结果与分析

6.3.1 优化前后各风机累积疲劳损伤对比

由于风机数量较多，并且相同风机不同元件累积疲劳损伤相差几个数量级，不同风机之间的累计疲劳损伤可能相差几个数量级，随机选取风机 1、11、63 台机组优化前后累积疲劳损伤随时间的增长曲线进行对比分析，分别如下图所示：

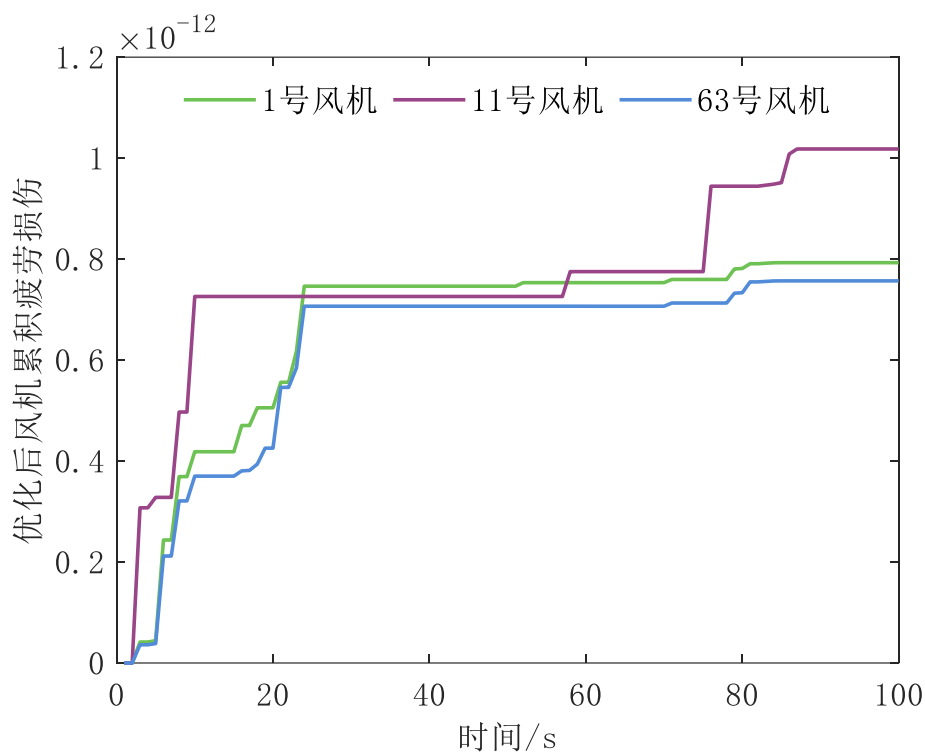


图 6.4 优化前风机累计疲劳损伤

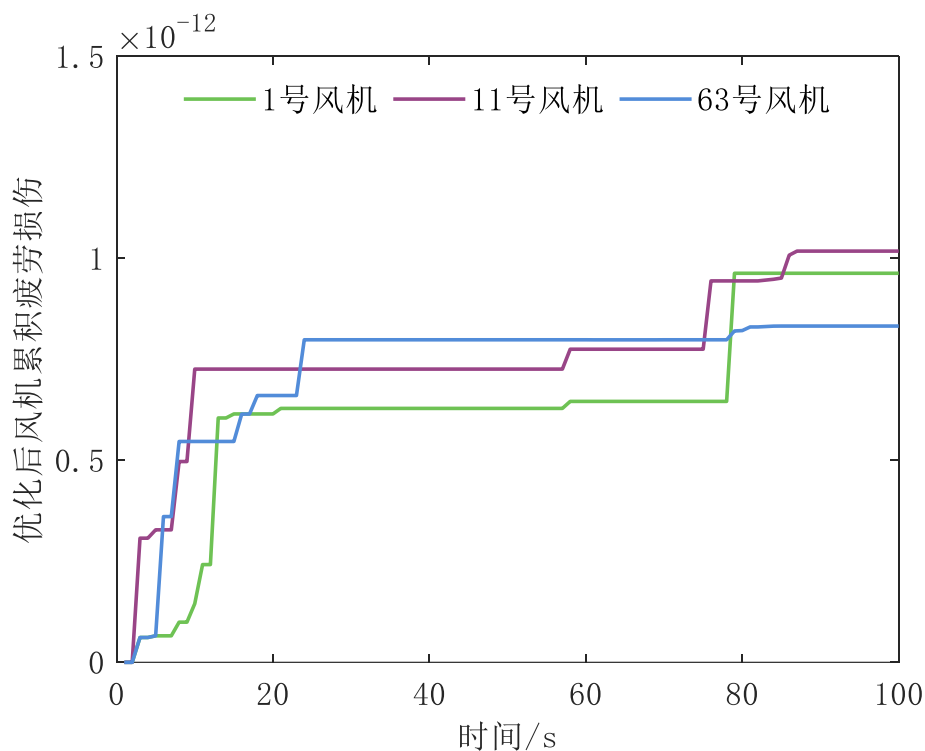


图 6.5 优化后风机累计疲劳损伤

从上述两幅图可以看出，三台风机优化后的累计疲劳损伤均小于优化前的累计疲劳损伤，有效降低了风机的疲劳损伤。还可以看出，由于风机本身特性的不同，采用不同的功率分配方法，各风机优化前后的损伤趋势一致，即风机 11 的疲劳损伤基本在其余两台机组之上。另外，由于本方法是对所有风机进行全局的优化，其余风机同样有不错的效果。

6.3.2 优化前后所有风机总累积疲劳损伤对比

将优化前后风机总累积疲劳损伤随时间的变化曲线画在同一张图中做对比，由于二者相差几个数量级，采用主次坐标轴进行作图，其中优化前曲线为左侧坐标轴，优化前曲线为右侧坐标轴。

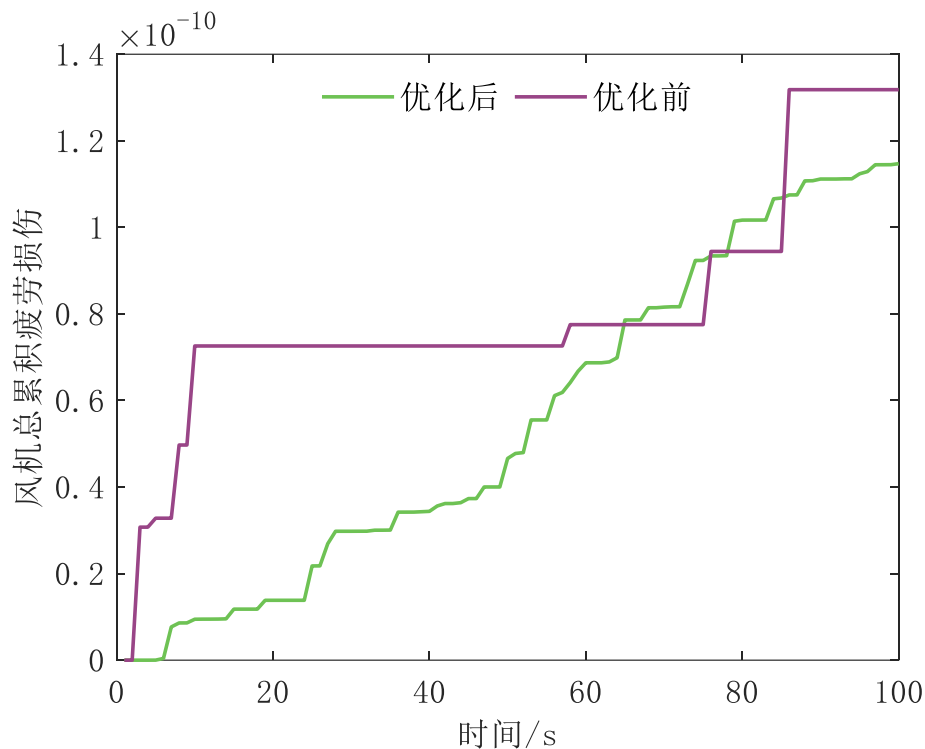


图 6.6 优化前后所有风机总累计疲劳损伤

从图中可以看出，风机优化后总疲劳损伤基本小于优化前总疲劳损伤，总体下降约 10%，优化效果较好。同时，优化前的累积疲劳损伤由于是直接调度指令平均分配到各风机，由于总的功率波动不大，除以风机数量（100）后的功率变化更小，从而导致主轴扭矩和塔架推力波动不大，进而有一段时间保持不变，直到出现调度总功率波动较大时才出现较大的累积疲劳损伤。而优化后由于是采取实时的强化学习优化算法，故进行了实时的更新。

6.3.3 优化前后各风机功率参考方差对比

将优化前后各风机 100 个时刻参考功率的方差画在同一张图中做对比如下图所示。

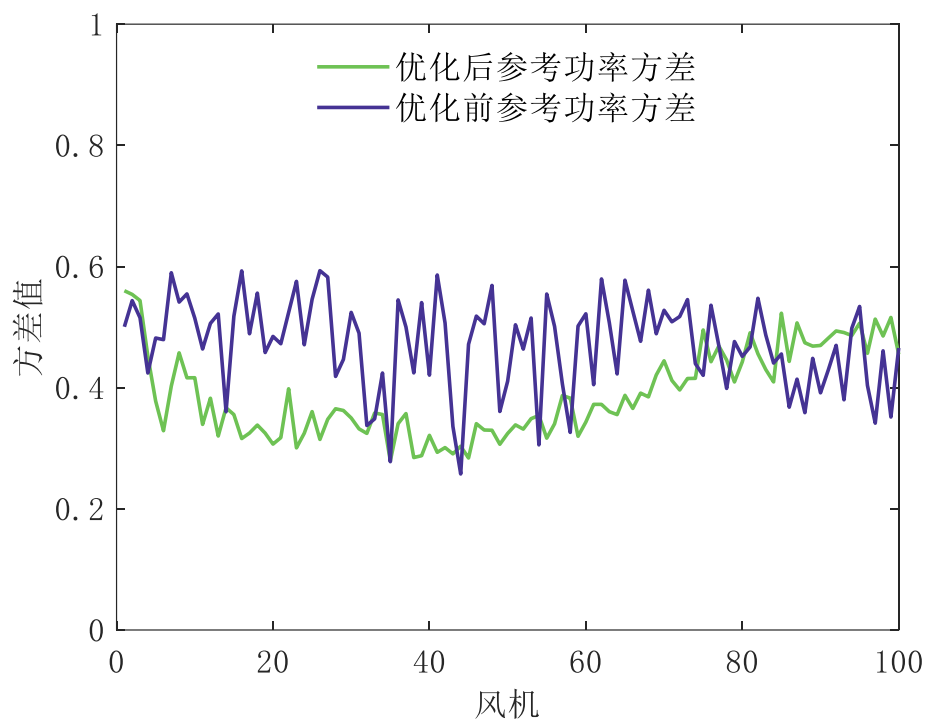


图 6.7 优化前后各风机参考功率方差

从上图可以看出，100 台风机中，优化后参考功率方差大部分小于优化前的，方差降低约 30%，总体上看优化后参考功率的波动性较小，总疲劳损伤较小，与之前的分析保持一致。

6.3.4 实时风机功率分配动图

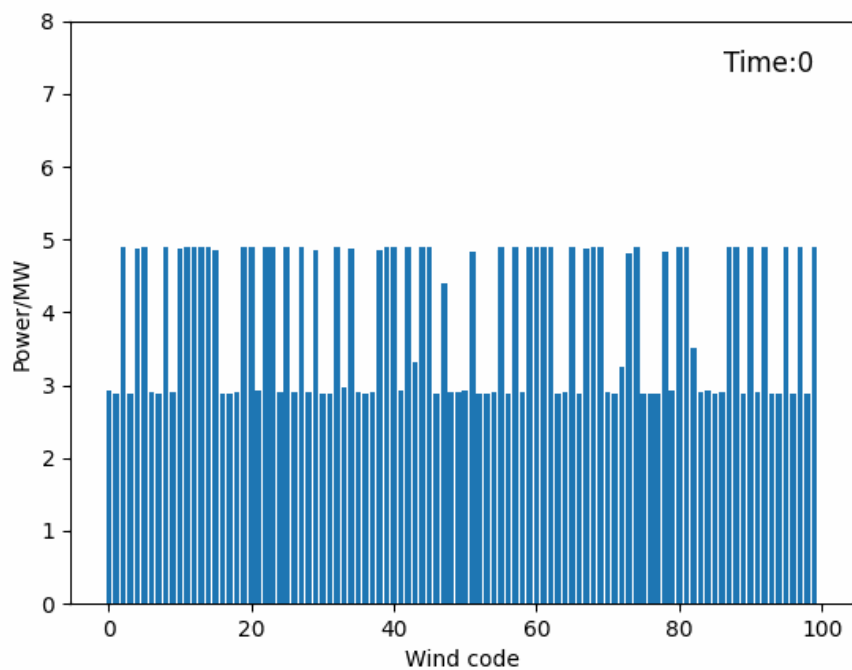


图 6.8 实时风机功率分配动图

如图所示，所提算法可以每个时刻实时分配功率，尽可能通过每个时刻的调整优化减缓疲劳损伤的累积。

七、问题四：模型建立与求解

7.1 模型建立

7.1.1 基于 CNN-LSTM 的时序数据滚动预测

对于风电场调度指令和风速测量的信息阻塞问题，涉及到信息阻塞的判断与阻塞数据的再生成。本节旨在通过引入负预测算法，将阻塞的不确定性问题转换为负荷功率/风速预测算法的预测精确性问题，这是由于负荷需求通常具有日周期性，且对于超短期风速具有一定的趋势特征。理想情况下，若数据预测完全精确，则相当于不存在信息阻塞。

首先采用神经网络对风速进行时序预测，具体而言，先通过历史数据去训练捕获风电场功率优化任务所需数据的时序关联。本文取本时刻之前的 10 个历史数据作为输入，当前时刻数据作为输出，训练神经网络学习。应用时，若检测到当前采集到的数据与前一时刻完全一致，说明发生了传输的延时，使用历史时刻数据输入到训练好的神经网络之中，即可预测当前时刻的数据。

本文采用长短期记忆网络(Long-short term memory, LSTM) 和卷积神经网络(Convolutional Neural Network, CNN) 相结合的方式进行时序预测。LSTM 能够实现对远距离信息的有效控制，较好的解决梯度消失与梯度爆炸问题，其结构如下所示：

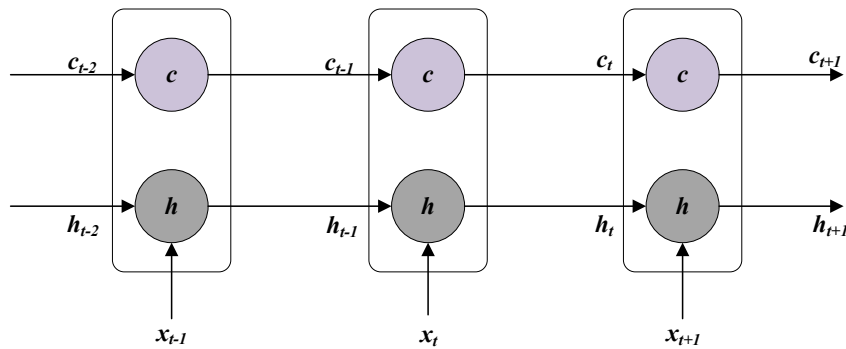


图 7.1 LSTM 网络结构模型

CNN 通常采用反向传播算法进行训练，并形成了三层结构的前馈神经网络，并且也同样具备了局部连接和权重共享，因此也赋予了该算法在进行部分动作时的稳定性。CNN 一般包括输入层、隐含层和输出层，其中隐含层又包括卷积层、池化层和全连接层。结构如图 7.2 所示。

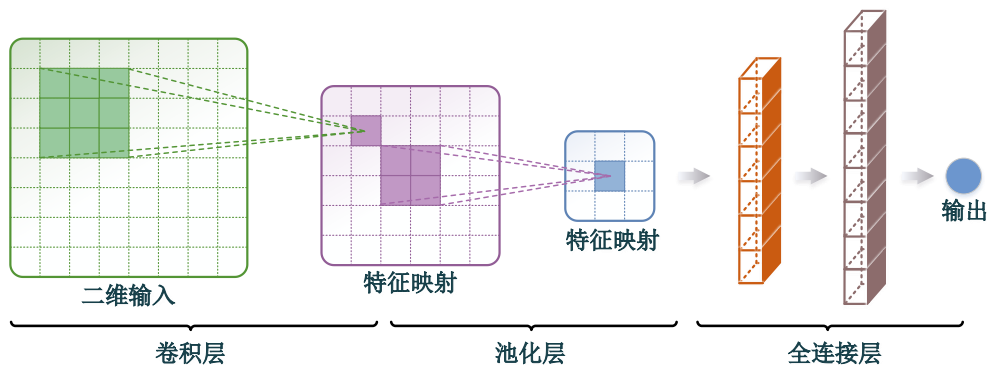


图 7.2 CNN 网络结构模型

本文基于卷积-长短期记忆网络(CNN-LSTM)的深度学习模型提取风电场本地数据的时序关系，实现对通信延迟点的时序预测。采用附件 3 中 2000 组历史数据拟合数据的时序关系，首先将原始数据序列输入 LSTM 模型，让模型提取序列的时序特征；再将选取的

特征输入全连接层中做特征映射，通过特征分离的方式进一步序列的非线性关系；最后，采用 Python 中的拼接函数 Concatenate 将 CNN、LSTM 以及特征分离模块提取到的特征进行拼接，并将拼接所得结果输入到后续的全连接层中以便模型能同时学习各模块提取的特征，达到特征融合的目的，进而实现阻塞数据的滚动预测。

7.1.2 基于鲁棒优化的风机功率分配模型

基于鲁棒优化的风机功率分配模型分为第一阶段主问题和第二阶段子问题，主问题目标函数为：

$$\min \sum_{t=1}^{T_{all}} \sum_{i=1}^{N_w} (\varepsilon_1 \cdot D_{t,i}^T + (1-\varepsilon_1) \cdot D_{t,i}^F) \quad (7.1)$$

子问题目标函数为：

$$\max \sum_{t=1}^{T_{all}} \sum_{i=1}^{N_w} (\varepsilon_1 \cdot D_{t,i}^T + (1-\varepsilon_1) \cdot D_{t,i}^F) \quad (7.2)$$

式中： $D_{t,i}^T$ 为归一化后主轴累积疲劳损伤； $D_{t,i}^F$ 为归一化后塔架累积疲劳损伤；归一化的原因因在于 $D_{t,i}^T$ 与 $D_{t,i}^F$ 是两个不同的类型量，实际值相差几个数量级，进行多目标优化的时候影响较大，进行归一化操作以便优化，放缩 4 个量级； T_{all} 为优化的时段，本文中为 100 个时刻，之所以使用总的优化时段，是因为如果只对当前时刻进行优化，只关注了当前的信息，而忽略了过去、未来的信息，容易陷入局部最优解，因此直接从全局进行优化，保证总的时段累积疲劳损伤最小； N_w 为风机总数量，本文中为 100 台风机； ε_1 为目标权重，两个目标权重和为 1，由于进行了归一化，本文直接取 0.5。

约束条件包括功率平衡约束：

$$\sum_{i=1}^n P_{wt,i}^{ref} = P_{wf}^{ref} \quad (7.3)$$

式中： P_{wf}^{ref} 为调度下达到给风电场的功率值， $P_{wt,i}^{ref}$ 为各台风机的分配功率。

风机功率上下限约束：

$$P_{wt,i}^{min} \leq P_{wt,i}^{ref} \leq P_{wt,i}^{max} \quad (7.4)$$

式中： $P_{wt,i}^{min}$ 、 $P_{wt,i}^{max}$ 分别为当前风速下的最小最大值。

风机功率偏移约束：

$$|P_{wt,i}^{ref} - P_{wt,i}^{av}| \leq 1 \quad (7.5)$$

式中： $P_{wt,i}^{av}$ 为平均功率分配下的指标，优化的功率 $P_{wt,i}^{ref}$ 与其偏移不超过 1MW。

在对问题二的解决中，已经建立了风机功率参考值 $P_{wt,i}^{ref}$ 与主轴扭矩 T_m 、塔架推力 F_m 的关系，主要有如下公式：

$$P_o^t = k_1 P_{ref}^{t-1} + k_2 v_t^3 + k_3 \quad (7.6)$$

$$P_m^t = P_o^t / \eta \quad (7.7)$$

$$T_m^t = P_m^t / \omega_r \quad (7.8)$$

$$F_m^t = P_m^t / v(1-a) \quad (7.9)$$

式中： k_1 、 k_2 、 k_3 、 a 为通过最小二乘法估计的已知数， η 为给定的功率转化效率， ω_r 、 v 为给定的状态量，若发现了通信延时，则使用 CNN-LSTM 进行时序预测得到新的风速 v 。

为了降低模型求解难度，需要对主轴扭矩 T_m 、塔架推力 F_m 与主轴累积疲劳损伤 $D_{t,i}^T$ 、塔架累积疲劳损伤 $D_{t,i}^F$ 的关系进行一定简化，以满足实时求解的要求，本节认为主轴累积

疲劳损伤增加值 $\Delta D_{t,i}^T$ 、塔架累积疲劳损伤增加值 $\Delta D_{t,i}^F$ 与主轴扭矩 T_m 、塔架推力 F_m 对相应基准值的变化量成线性关系，即：

$$\Delta D_{t,i}^T = \frac{(T_{t,i} - T_t^{base})}{T_t^{base}} \quad (7.10)$$

$$\Delta D_{t,i}^F = \frac{(F_{t,i} - F_t^{base})}{F_t^{base}} \quad (7.11)$$

风速 v 不超过测量风速 v_0 的 10%，如下式所示：

$$0.9v_0 \leq v \leq 1.1v_0 \quad (7.12)$$

7.2 模型求解

鲁棒优化依靠主问题和子问题不断迭代求解，首先，给定初始场景下风速 v ，求出主问题最优解 P_{wf}^{ref} ；子问题在 P_{wf}^{ref} 给定下求其目标值，即目标函数最恶劣条件的风速 v ；继续反代回主问题，不断迭代求解，直至收敛。

7.3 求解结果与分析

优化前后各风机累积疲劳损伤对比：

随机选取风机 1、11、63 台机组优化前后累积疲劳损伤随时间的增长曲线进行对比分析，分别如下图所示：

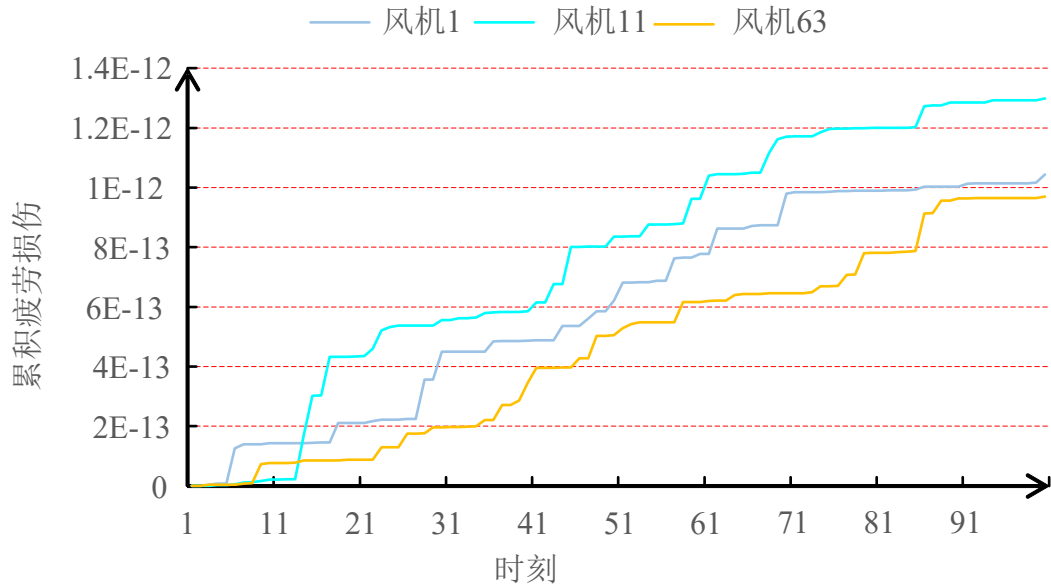


图 7.3 优化后风机累计疲劳损伤

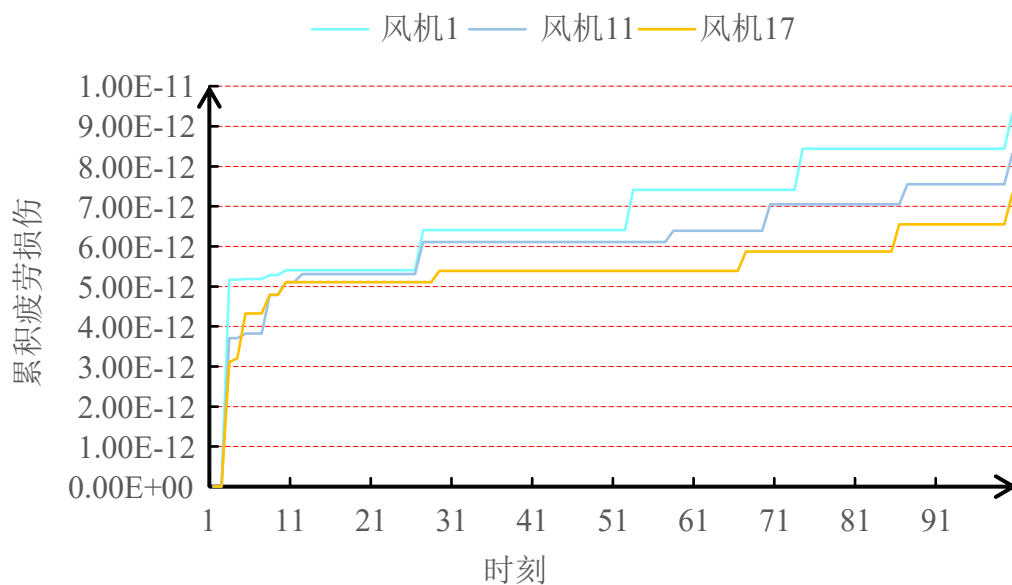


图 7.4 优化前风机累计疲劳损伤

从上述两幅图可以看出，添加噪声后的优化，模型优化的稳定性大幅下降，在小时段中开始出现较高的损伤，进而在累计疲劳损伤中表现出较为连续的上升曲线。但总体的损伤指标要优于优化前指标。

优化前后所有风机总累积疲劳损伤对比：

优化前后风机总累积疲劳损伤随时间的变化曲线展示如下：

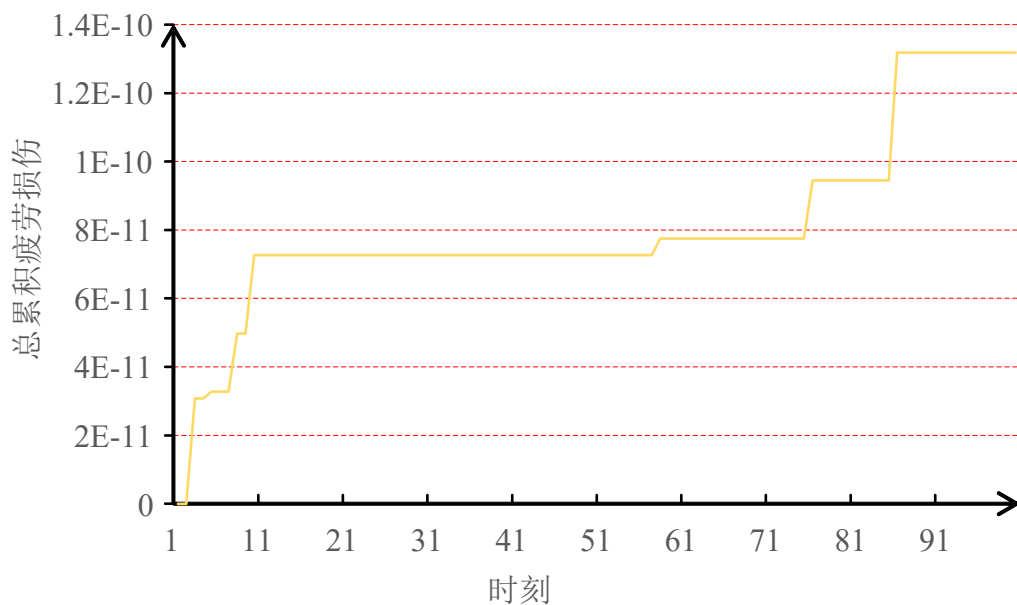


图 7.5 优化后所有风机总累积疲劳损伤

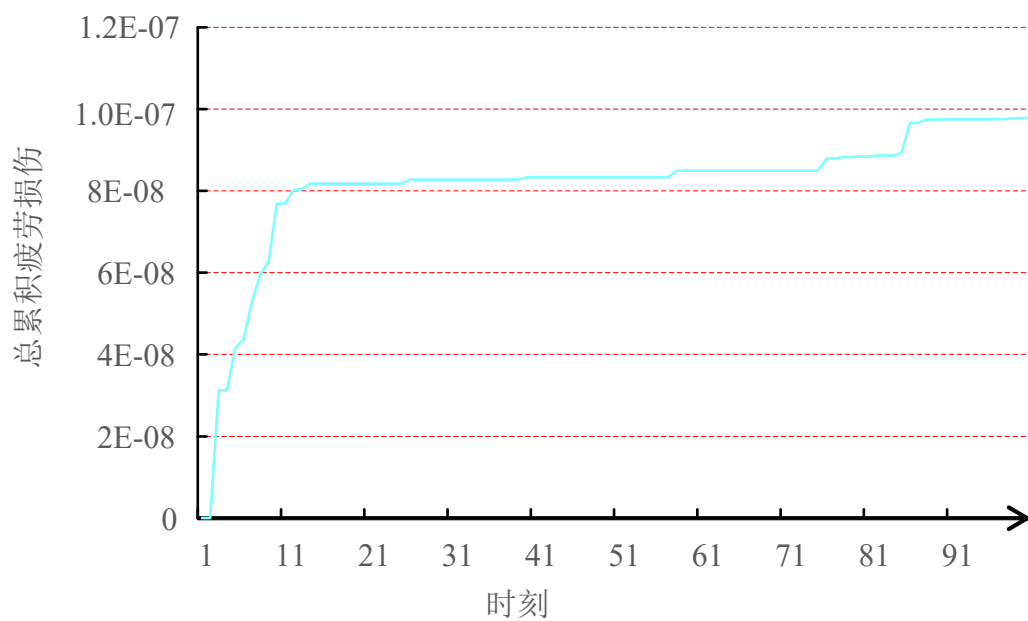


图 7.6 优化前所有风机总累计疲劳损伤

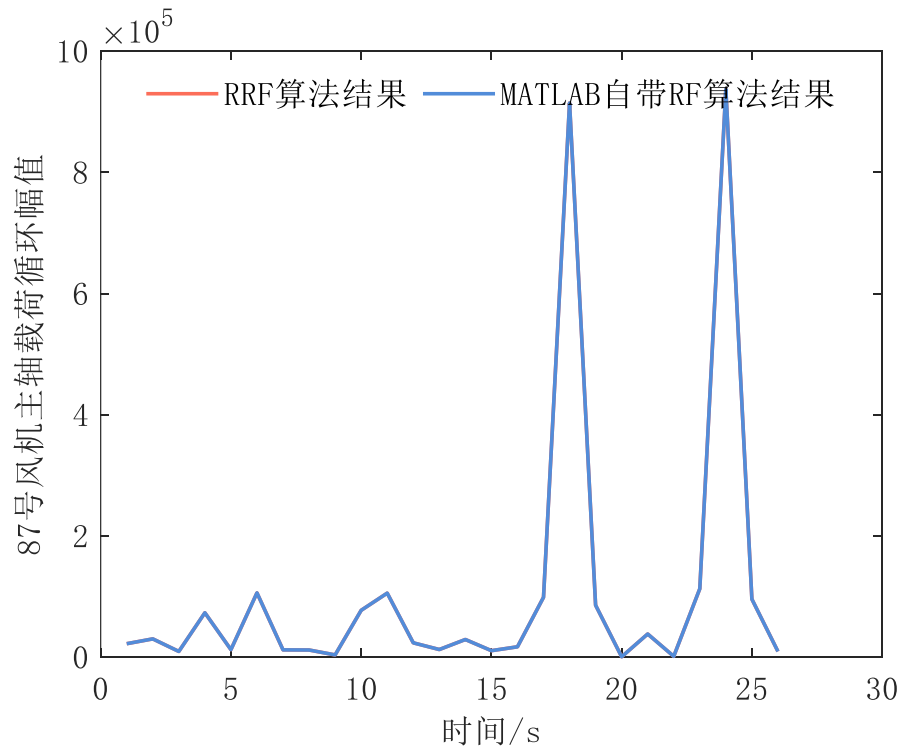
此时可以观察到优化后的累计数值连续性降低，这说明了叠加的随机噪声可能具有零均值特性，因此当不同机组求和后，这些随机噪声带来的影响在一定程度上被抵消。

八、模型评价与改进

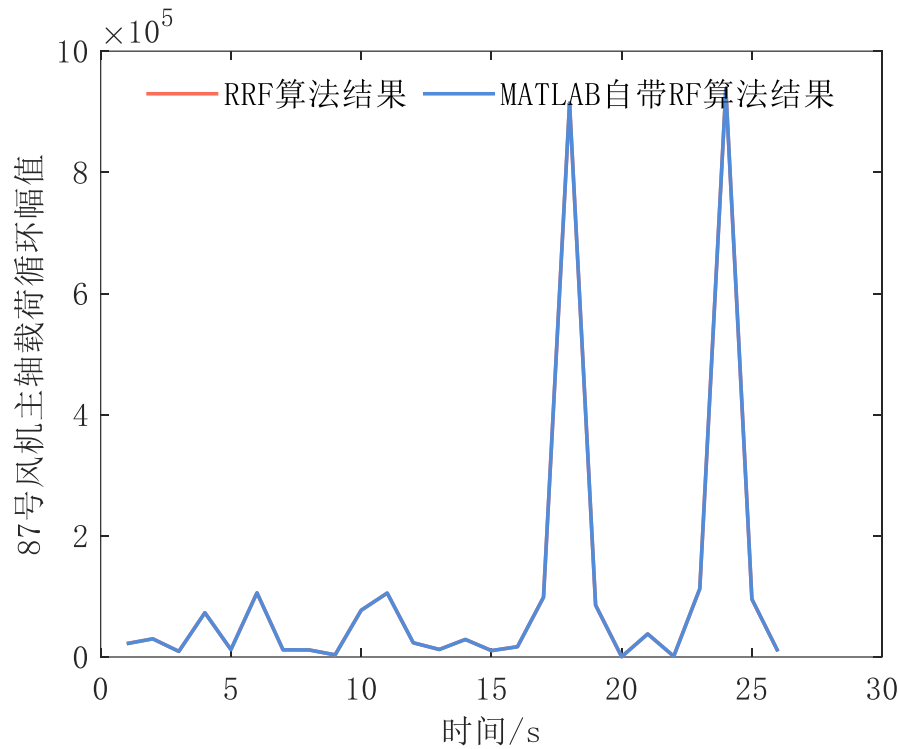
8.1 问题一模型优缺点分析

本节对问题 1 模型优缺点进行分析，主要分为 RRF 算法与 RNCFD 指标，首先分析 RRF 算法的优缺点，先验证 RRF 算法的有效准确性，通过 MATLAB 自带 `rainflow` 函数直接计算 100s 的随机载荷谱提取载荷循环，然后与通过本文提出的 RRF 算法从第 1s 计算到第 100s 时计算出的载荷循环进行对比，分析 RRF 算法的有效性，主要对比载荷循环的幅值与均值，为具有典型性，选取 87 号风机主轴的随机载荷谱数据进行计算比较分析，具体结果见图 8.1。

可以发现本文所提 RRF 算法计算所得载荷循环幅值、均值与 MATLAB 自带 `rainflow` 函数计算所得载荷循环幅值、均值完全重合，说明所提 RRF 算法能够准确计算随机载荷谱的载荷循环，验证了 RRF 算法的有效性。



(a) 87 号风机主轴载荷循环幅值对比



(b) 87 号风机主轴载荷循环均值对比

图 8.1 RRF 算法有效性验证

RRF 算法能够拥有与 RF 算法相同的计算准确度，并且其能够计算一段时间的每个时刻载荷循环，实时计算载荷循环，相较于 RF 算法具有实时性上的优越性。

其次对于 RRF 算法的缺陷，即其需要计算完所有时刻载荷数据才能提取出最大载荷循环，最大载荷循环的损伤影响在最后时刻才显现，该缺陷会影响累积疲劳损伤指标的实时性，从而产生误差，下面会通过比较 RNCFD 指标是否删去暂时损伤之间的误差，即是否将 RRF 算法预存库里的最大载荷循环纳入实时考虑，从而体现 RRF 无法实时考虑最大载荷循环的影响。同样采用 87 号风机主轴的随机载荷谱数据进行计算，比较 RNCFD 指标是否删去暂时损伤之间的误差，其结果见图 8.2。

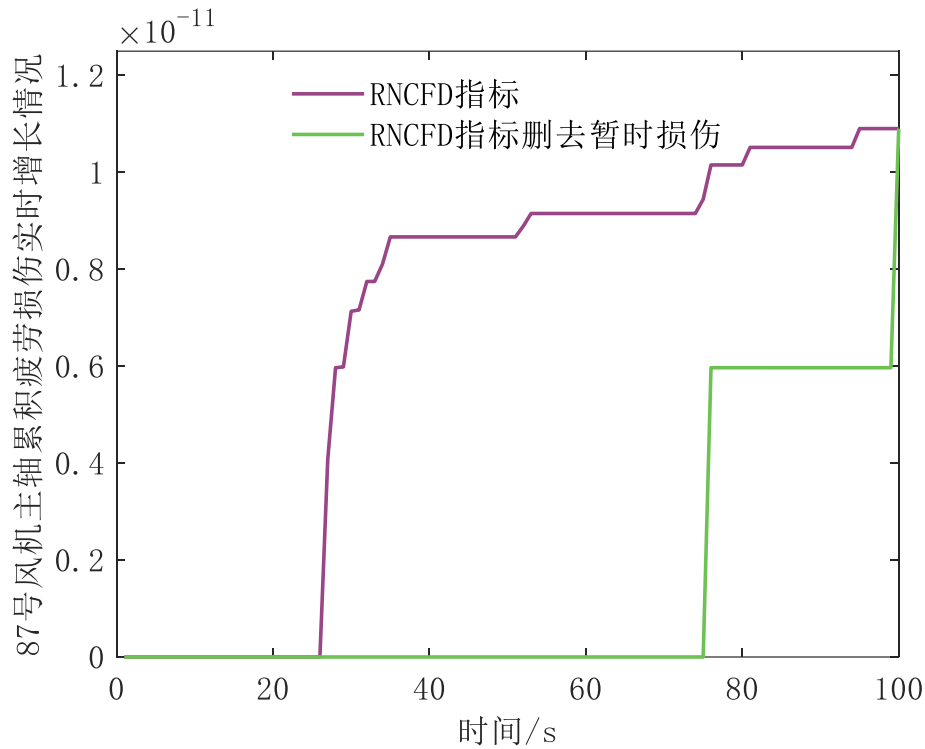


图 8.2 RNCFD 指标是否删去暂时损伤对比

分析图 8.2 可以发现，RNCFD 指标删去暂时损伤之后就难以实时考虑最大载荷循环的影响，最大载荷循环在最后时刻才提取出，导致第 99s 到第 100s 指标突变，并且实际载荷大波动在 27s 左右，RNCFD 指标删去暂时损伤无法准确判断大扰动的产生时间，导致指标实时性效果较差，因此为克服 RRF 算法难以实时考虑最大载荷循环的缺陷，通常可以考虑直接对算法改进或者对后续指标进行改进，本文采取的方法为改进后续指标，提升其实时性。

以上对问题 1 的模型的优缺点进行了介绍分析，后续更进一步对问题一算法的优化细节可以考虑提高实时算法计算效率，并考虑改进指标公式以提高准确性。

8.2 问题二模型优缺点分析

优点：考虑风电场从接受指令到实际控制的全过程，公式简单，计算方便。

缺点：精确度不足，采用端对端，内部参数较为粗糙。

8.3 问题三模型优缺点分析

优点：能以极快速度完成实时优化；决策考虑到了过去时刻累计疲劳损伤的变化特征以及未来的风电场环境变化趋势；由于采用强化学习，强化学习使用了神经网络，输出不确定性较大，可能会越限，本文采用神经网络内部进行约束+安全层解决该问题。

缺点：算法训练时间长，训练时间短会导致优化效果差，且训练稳定性差。

8.4 问题四模型优缺点分析

优点：通过预测方式填补阻塞值保证优化的连续性，将阻塞带来的不确定性转嫁为拟合误差，采用鲁棒优化能够在带噪声的不确定环境下最大程度上满足安全约束。

缺点：预测无法完全精准，鲁棒优化的策略过于保守。

九、结论

首先，针对雨流计数法无法实时在线求解的难题，本研究成功开发了实时雨流计数法，该算法在保证计算准确性的同时，实现了对载荷循环的实时快速计算。进一步地，为克服实时雨流计数法在关键时刻载荷反应不足及线性累积疲劳损伤理论的局限性，本研究引入了实时非线性累积疲劳损伤指标，该指标通过引入时间修正系数与暂时损伤，不仅有效考虑了载荷循环的顺序效应，还能实时识别响应最大载荷循环，为风机的累积疲劳损伤评估提供更为精细和动态的方法。

其次，针对塔架推力和主轴扭矩等关键数据难以直接测量的问题，本研究深入分析了风机输出功率响应、载荷变化延迟效应以及风速、功率、转速、桨距角之间的复杂关系，构建精确有效的载荷-参考功率拟合模型。利用非线性最小二乘算法迭代求解参数，确保了模型的准确性和实用性，为风电场的疲劳损伤计算提供了坚实的数据基础。

在实时功率分配方面，本文基于强化学习理论，提出了高效且精确的实时优化算法。该算法能够应对动态高维场景，充分考虑历史载荷曲线特征、未来风速及功率需求趋势，实现了风电场功率分配的实时精确优化。同时针对强化学习输出可能越限的问题，本文通过设计特殊神经网络层和放缩限幅策略，有效保证了功率平衡和机组功率约束的满足，同时对于极少数越限情况，设计了安全层进行矫正，确保了算法的可靠性。

最后，针对风电场环境数值、噪声干扰及数据阻塞等问题，本研究提出了综合解决策略。通过设置判断条件识别数据阻塞现象，并引入预测算法将不确定性问题转化为预测准确性问题。同时，构建鲁棒优化模型，通过迭代计算最差场景下的最优动作，确保模型在多数有效场景下既能满足所有约束条件，又能保持一定的优化效能，为风电场在复杂多变环境下提供了更为稳健与保守的功率分配方案。

十、参考文献

- [1]傅航杰,刘文业,李彦涌,等.实时雨流法及其在 IGBT 寿命预测中的应用研究[J].大功率变流技术,2017,(03):8-12.
- [2]吴林.基于自抗扰控制的风力发电机变速变桨控制系统研究[D].南京信息工程大学,2020.
- [3]林振福,杨铎炯.基于确定性策略梯度深度强化学习和模仿学习的多源微电网经济优化调度策略[J].电工技术,2023,(08):76-82.
- [4]丁奕程.考虑疲劳载荷的风电场多目标模型预测控制策略[D].东北电力大学,2024.
- [5]林振福,杨铎炯.基于确定性策略梯度深度强化学习和模仿学习的多源微电网经济优化调度策略[J].电工技术,2023,(08):76-82.

附录（代码及注释）

MATLAB 代码:

累积疲劳损伤实时计算程序:

%% 初始化并计时

clear;clc;

tic;

%% 导入初始数据

filename = '附件 1-疲劳评估数据.xls';

data1=table2array(readtable(filename, 'Sheet',2,'Range', 'B1:CW101')); %主轴载荷数据

data2=table2array(readtable(filename, 'Sheet',3,'Range', 'B1:CW101')); %塔架载荷数据

time1=size(data1,1); % 时间向量

%% 定义主轴扭矩存储库

initial_data1=cell(1,100); %定义主轴载荷数据极值存储库, 100 台风机

realrainflow_data1=cell(1,100); %定义实时雨流算法预存库 1 号, 100 台风机

realrainflow_data11=cell(1,100); %定义实时雨流算法预存库 11 号, 100 台风机

marker1=cell(1,100); %载荷极小极大值编号 (1 表示极大, 2 表示极小), 100 台风机

number1=cell(1,100); %极值存储库载荷数量计数, 100 台风机

realrainflow_data1_number=cell(1,100); %算法预存库载荷数量计数, 100 台风机

u1=cell(1,100);v1=cell(1,100);w1=cell(1,100); %各风机实时雨流计数法三点数据库, 100 台风机

cycle1number=cell(1,100); %载荷循环计数, 100 台风机

cycle1=cell(1,100); %风机载荷循环全半计数, 100 台风机

amplitude1=cell(1,100); %载荷循环幅值存储库, 100 台风机

meanvalue1=cell(1,100); %载荷循环均值存储库, 100 台风机

valueradius1=cell(1,100); %载荷循环起始时刻点存储库, 100 台风机

eqamplitude1=cell(1,100); %修正的载荷循环幅值存储库, 100 台风机

Ln1=cell(1,100); %等效疲劳载荷存储库, 100 台风机

N1=cell(1,100); %最大载荷循环次数存储库, 100 台风机

Df1=cell(1,100); %累积疲劳损伤存储库, 100 台风机

judge1=cell(1,100); %判断器, 判断载荷是否为极值, 100 台风机

%% 定义塔架推力存储库

initial_data2=cell(1,100); %定义塔架载荷数据极值存储库, 100 台风机

realrainflow_data2=cell(1,100); %定义实时雨流算法预存库 1 号, 100 台风机

realrainflow_data22=cell(1,100); %定义实时雨流算法预存库 11 号, 100 台风机

marker2=cell(1,100); %载荷极小极大值编号 (1 表示极大, 2 表示极小), 100 台风机

number2=cell(1,100); %极值存储库载荷数量计数, 100 台风机

```

realrainflow_data2_number=cell(1,100); %算法预存库载荷数量计数, 100 台风机
u2=cell(1,100);v2=cell(1,100);w2=cell(1,100); %各风机实时雨流计数法三点数据库, 100 台风机
cycle2number=cell(1,100); %载荷循环计数, 100 台风机
cycle2=cell(1,100); %风机载荷循环全半计数, 100 台风机
amplitude2=cell(1,100); %载荷循环幅值存储库, 100 台风机
meanvalue2=cell(1,100); %载荷循环均值存储库, 100 台风机
valueradius2=cell(1,100); %载荷循环起始时刻点存储库, 100 台风机
eqamplitude2=cell(1,100); %修正的载荷循环幅值存储库, 100 台风机
Ln2=cell(1,100); %等效疲劳载荷存储库, 100 台风机
N2=cell(1,100); %最大载荷循环次数存储库, 100 台风机
Df2=cell(1,100); %累积疲劳损伤存储库, 100 台风机
judge2=cell(1,100); %判断器, 判断载荷是否为极值, 100 台风机

%% 实时计算, 从时刻 3 开始
for i=3:time1
    %% 各风机计算当前主轴扭矩累计疲劳损伤
    for j=1:100
        if i==3
            %% 开始计算主轴扭矩累计疲劳损伤
            % 首先将各风机时刻 1, 2 的载荷数据放入极值存储库, 时刻 1 必为极值, 时刻 2 需要后续判断

            % 主轴
            if data1(1,j)<=data1(2,j)
                marker1{j}(1)=2;
                marker1{j}(2)=1;
            else
                marker1{j}(1)=1;
                marker1{j}(2)=2;
            end
            initial_data1{j}(marker1{j}(1),1)=data1(1,j);
            initial_data1{j}(3,1)=1;
            initial_data1{j}(marker1{j}(2),2)=data1(2,j);
            initial_data1{j}(3,2)=2;
            % 将各风机极值存储库时刻 1,2 的载荷数据导入算法预存库
            realrainflow_data1{j}(1,1)=initial_data1{j}(marker1{j}(1),1); %时刻 1 的载荷数据值导入算法预存库
            realrainflow_data1{j}(2,1)=initial_data1{j}(3,1); %时刻 1 的载荷数据位置导入算法预存库
            realrainflow_data1{j}(1,2)=initial_data1{j}(marker1{j}(2),2); %时刻 2 的载荷数据值导入算法预存库
            realrainflow_data1{j}(2,2)=initial_data1{j}(3,2); %时刻 2 的载荷数据位置导入算法预存库
        end
    end
end

```



```

realrainflow_data1_number{j}(1)=size(realrainflow_data1{j}(1,1:end),2); %各风机算法
预存库载荷数量
    judge1{j}=0; %判断各风机极值存储库最新载荷是否替换过的标记
    number1{j}=2; %各风机极值存储库计数，从 2 开始
    cycle1number{j}(1)=0; %各风机实际载荷循环计数,从 0 开始
    cycle1number{j}(2)=0; %各风机暂时载荷循环计数,从 0 开始
end

% 判断各风机极值存储库前一时刻载荷是否为极值
if
(initial_data1{j}(marker1{j}(number1{j}),number1{j})-initial_data1{j}(marker1{j}(
number1{j}-1),number1{j}-1))*...
    (initial_data1{j}(marker1{j}(number1{j}),number1{j})-data1(i,j))<0
    % 前一时刻载荷不为极值，当前时刻载荷替换各风机极值存储库的最新载荷
    judge1{j}=1;
    initial_data1{j}(marker1{j}(number1{j}),number1{j})=data1(i,j); %当前时
刻载荷替换各风机极值存储库的最新载荷
    initial_data1{j}(3,number1{j})=i; %替换后的当前时刻载荷的时刻点
else
    % 前一时刻载荷为极值，当前时刻载荷的时刻点导入各风机极值存储库

marker1{j}(number1{j}+1)=(-1)^(marker1{j}(number1{j})-1)+marker1{j}(number1{j});
    initial_data1{j}(marker1{j}(number1{j}+1),number1{j}+1)=data1(i,j); %当
前时刻载荷的时刻点导入各风机极值存储库
    initial_data1{j}(3,number1{j}+1)=i; %当前时刻载荷的时刻点
    number1{j}=number1{j}+1; %各风机极值存储库计数
end

% 各风机极值存储库最新载荷是否替换过
if judge1{j}==0
    %未替换过，各风机极值存储库的最新载荷导入各风机算法预存库

realrainflow_data1{j}(1,end+1)=initial_data1{j}(marker1{j}(number1{j}),number1{j}
); %各风机极值存储库最新载荷导入各风机算法预存库
    realrainflow_data1{j}(2,end)=initial_data1{j}(3,number1{j}); %各风机算法
预存库里最新载荷为哪一时刻的载荷

realrainflow_data1_number{j}(1)=size(realrainflow_data1{j}(1,1:end),2); %各风机算法
预存库载荷数量
else
    %替换过，各风机极值存储库的最新载荷替换各风机算法预存库的最新载荷
    judge1{j}=0; %重置判断标记

```

```

realrainflow_data1{j}(1,end)=initial_data1{j}(marker1{j}(number1{j}),number1{j});
%各风机极值存储库的最新载荷替换各风机算法预存库的最新载荷
    realrainflow_data1{j}(2,end)=initial_data1{j}(3,number1{j}); %各风机算法
    预存库里最新载荷为哪一时刻的载荷
end

% 判断各风机算法预存库里载荷数量是否大于等于 3 个
while realrainflow_data1_number{j}(1)>=3
    % 各风机算法预存库最新 3 个载荷作为实时雨流计数法三点数据
    u1{j}=realrainflow_data1{j}(1,end-2);
    v1{j}=realrainflow_data1{j}(1,end-1);
    w1{j}=realrainflow_data1{j}(1,end);
    % 比较相邻两个点范围大小
    if abs(v1{j}-u1{j})<abs(v1{j}-w1{j})
        cycle1number{j}(1)=cycle1number{j}(1)+1; %各风机实际载荷循环计数
        % 各风机算法预存库载荷数量是否等于 3，进行全循环半循环判断
        if realrainflow_data1_number{j}(1)==3
            cycle1{j}(cycle1number{j}(1),1)=0.5;
            amplitude1{j}(cycle1number{j}(1),1)=abs(v1{j}-u1{j}); %各风机实
            际载荷循环幅值
            meanvalue1{j}(cycle1number{j}(1),1)=(u1{j}+v1{j})/2; %各风机实际
            载荷循环均值
            valueradius1{j}(cycle1number{j}(1),1)=realrainflow_data1{j}(2,end-2); %各风机实际载
            荷循环起点时刻
            valueradius1{j}(cycle1number{j}(1),2)=realrainflow_data1{j}(2,end-1); %各风机实际载
            荷循环终点时刻
            realrainflow_data1{j}(:,end-2)=[];

            realrainflow_data1_number{j}(1)=size(realrainflow_data1{j}(1,1:end),2); %算法预存
            库里载荷数量
        else
            cycle1{j}(cycle1number{j}(1),1)=1;
            amplitude1{j}(cycle1number{j}(1),1)=abs(v1{j}-u1{j}); %各风机实
            际载荷循环幅值
            meanvalue1{j}(cycle1number{j}(1),1)=(u1{j}+v1{j})/2; %各风机实际
            载荷循环均值
            valueradius1{j}(cycle1number{j}(1),1)=realrainflow_data1{j}(2,end-2); %各风机实际载
            荷循环起点时刻

```

```

valueradius1{j}(cycle1number{j}(1),2)=realrainflow_data1{j}(2,end-1); %各风机实际载
荷循环终点时刻

        realrainflow_data1{j}(:,end-2)=[];
        realrainflow_data1{j}(:,end-1)=[];

realrainflow_data1_number{j}(1)=size(realrainflow_data1{j}(1,1:end),2); %算法预存库
里载荷数量

        end
    else
        break;
    end
end
% 各风机实际载荷循环计算实际累积疲劳损伤
if cycle1number{j}(1)>=1

eqamplitude1{j}(1:cycle1number{j}(1),1)=amplitude1{j}(1:cycle1number{j}(1),1)./(1-
meanvalue1{j}(1:cycle1number{j},1)./50000000); %修正载荷循环，修正为均值为0的载荷循环
幅值

Ln1{j}(i,1)=sum((eqamplitude1{j}(1:cycle1number{j}(1),1).^10).*cycle1{j}(1:cycle1n
umber{j}(1),1),1)./42565440.4361;

N1{j}(1:cycle1number{j}(1),1)=9.77*10^70./(eqamplitude1{j}(1:cycle1number{j}(1),1)
.^10); %S-N 曲线判断同一载荷循环幅值的最大循环载荷次数

Df1{j}(i,1)=sum((cycle1{j}(1:cycle1number{j}(1),1)./N1{j}(1:cycle1number{j}(1),1))
.^(1-(exp(-valueradius1{j}(1:cycle1number{j}(1),1)./5))./40),1); %线性累积损伤理论计
算实际累积疲劳损伤值

    end

    % 计算最后实际以及暂时载荷循环
    realrainflow_data11{j}=realrainflow_data1{j};
    realrainflow_data1_number{j}(2)=size(realrainflow_data11{j}(1,1:end),2);
    if i==time1
        while realrainflow_data1_number{j}(2)>=2
            cycle1number{j}(2)=cycle1number{j}(2)+1; %载荷循环计数
            cycle1{j}(cycle1number{j}(2),2)=0.5;

            amplitude1{j}(cycle1number{j}(2),2)=abs(realrainflow_data11{j}(1,1)-realrainflow_d
            ata11{j}(1,2));

            meanvalue1{j}(cycle1number{j}(2),2)=(realrainflow_data11{j}(1,1)+realrainflow_data
            11{j}(1,2))/2;

```

```

        valueradius1{j}(cycle1number{j}(2),3)=realrainflow_data11{j}(2,1);
        valueradius1{j}(cycle1number{j}(2),4)=realrainflow_data11{j}(2,2);
        realrainflow_data11{j}(:,1)=[];

    realrainflow_data1_number{j}(2)=size(realrainflow_data11{j}(1,1:end),2);
    end
    % 各风机实际载荷循环计算实际累积疲劳损伤
    if cycle1number{j}(2)>=1

    eqamplitude1{j}(1:cycle1number{j}(2),2)=amplitude1{j}(1:cycle1number{j}(2),2)./(1
    -meanvalue1{j}(1:cycle1number{j}(2),2)./50000000); %修正载荷循环,修正为均值为0的载
    荷循环幅值

    Ln1{j}(i,2)=sum((eqamplitude1{j}(1:cycle1number{j}(2),2).^10).*cycle1{j}(1:cycle1
    number{j}(2),2),1)./42565440.4361;

    N1{j}(1:cycle1number{j}(2),2)=9.77*10^70./(eqamplitude1{j}(1:cycle1number{j}(2),2
    ).^10); %S-N 曲线判断同一载荷循环幅值的最大循环载荷次数

    Df1{j}(i,2)=sum((cycle1{j}(1:cycle1number{j}(2),2)./N1{j}(1:cycle1number{j}(2),2)
    ).^(1-(exp(-valueradius1{j}(1:cycle1number{j}(2),3)./5))./40),1); %非线性累积损伤理
    论计算暂时累积疲劳损伤值
    end
    else
        while realrainflow_data1_number{j}(2)>=2
            cycle1number{j}(2)=cycle1number{j}(2)+1; %载荷循环计数
            cycle1{j}(cycle1number{j}(2),2)=0.5;

            amplitude1{j}(cycle1number{j}(2),2)=abs(realrainflow_data11{j}(1,1)-realrainflow_
            data11{j}(1,2));

            meanvalue1{j}(cycle1number{j}(2),2)=(realrainflow_data11{j}(1,1)+realrainflow_dat
            a11{j}(1,2))/2;
            valueradius1{j}(cycle1number{j}(2),3)=realrainflow_data11{j}(2,1);
            valueradius1{j}(cycle1number{j}(2),4)=realrainflow_data11{j}(2,2);
            realrainflow_data11{j}(:,1)=[];

            realrainflow_data1_number{j}(2)=size(realrainflow_data11{j}(1,1:end),2);
            end
            % 各风机暂时载荷循环计算暂时累积疲劳损伤
            if cycle1number{j}(2)>=1

            eqamplitude1{j}(1:cycle1number{j}(2),2)=amplitude1{j}(1:cycle1number{j}(2),2)./(1

```

```

-meanvalue1{j}(1:cycle1number{j}(2),2)./50000000); %修正载荷循环，修正为均值为0的载
荷循环幅值

Ln1{j}(i,2)=sum((eqamplitude1{j}(1:cycle1number{j}(2),2).^10).*cycle1{j}(1:cycle1
number{j}(2),2),1)./42565440.4361;

N1{j}(1:cycle1number{j}(2),2)=9.77*10^70./(eqamplitude1{j}(1:cycle1number{j}(2),2
).^10); %S-N 曲线判断同一载荷循环幅值的最大循环载荷次数

Df1{j}(i,2)=sum((cycle1{j}(2:cycle1number{j}(2),2)./N1{j}(2:cycle1number{j}(2),2
),1)+(cycle1{j}(1,2)./N1{j}(1,2)).^(1-(exp(-valueradius1{j}(1,3)./5))./40)); %非线性
累积损伤理论计算暂时累积疲劳损伤值

    end
    realrainflow_data11{j}=[];
    amplitude1{j}(1:cycle1number{j}(2),2)=0;
    meanvalue1{j}(1:cycle1number{j}(2),2)=0;
    valueradius1{j}(1:cycle1number{j}(2),3)=0;
    valueradius1{j}(1:cycle1number{j}(2),4)=0;
    cycle1number{j}(2)=0;
end
% 计算总累积疲劳损伤
if cycle1number{j}(1)>=1
    Lnall1(i,j)=(Ln1{j}(i,1)+Ln1{j}(i,2))^(1/10);
    Dfall1(i,j)=Df1{j}(i,1)+Df1{j}(i,2);
else
    Lnall1(i,j)=(Ln1{j}(i,2))^(1/10);
    Dfall1(i,j)=Df1{j}(i,2);
end

end

%% 各风机计算当前塔架推力累计疲劳损伤
for j=1:100
    if i==3
        %% 开始计算塔架推力累计疲劳损伤
        % 首先将各风机时刻 1, 2 的载荷数据放入极值存储库，时刻 1 必为极值，时刻 2 需要后
        续判断

        % 塔架
        if data2(1,j)<=data2(2,j)
            marker2{j}(1)=2;

```

```

        marker2{j}(2)=1;
    else
        marker2{j}(1)=1;
        marker2{j}(2)=2;
    end
    initial_data2{j}(marker2{j}(1),1)=data2(1,j);
    initial_data2{j}(3,1)=1;
    initial_data2{j}(marker2{j}(2),2)=data2(2,j);
    initial_data2{j}(3,2)=2;
    % 将各风机极值存储库时刻 1,2 的载荷数据导入算法预存库
    realrainflow_data2{j}(1,1)=initial_data2{j}(marker2{j}(1),1); %时刻 1 的
    载荷数据值导入算法预存库
    realrainflow_data2{j}(2,1)=initial_data2{j}(3,1); %时刻 1 的载荷数据位置导
    入算法预存库
    realrainflow_data2{j}(1,2)=initial_data2{j}(marker2{j}(2),2); %时刻 2 的
    载荷数据值导入算法预存库
    realrainflow_data2{j}(2,2)=initial_data2{j}(3,2); %时刻 2 的载荷数据位置导
    入算法预存库

    realrainflow_data2_number{j}(1)=size(realarainflow_data2{j}(1,1:end),2); %各风机算法
    预存库载荷数量

    judge2{j}=0; %判断各风机极值存储库最新载荷是否替换过的标记
    number2{j}=2; %各风机极值存储库计数, 从 2 开始
    cycle2number{j}(1)=0; %各风机实际载荷循环计数, 从 0 开始
    cycle2number{j}(2)=0; %各风机暂时载荷循环计数, 从 0 开始
end

% 判断各风机极值存储库前一时刻载荷是否为极值
if
(initial_data2{j}(marker2{j}(number2{j}),number2{j})-initial_data2{j}(marker2{j}(
number2{j}-1),number2{j}-1))*...
    (initial_data2{j}(marker2{j}(number2{j}),number2{j})-data2(i,j))<0
    % 前一时刻载荷不为极值, 当前时刻载荷替换各风机极值存储库的最新载荷
    judge2{j}=1;
    initial_data2{j}(marker2{j}(number2{j}),number2{j})=data2(i,j); %当前时
    刻载荷替换各风机极值存储库的最新载荷
    initial_data2{j}(3,number2{j})=i; %替换后的当前时刻载荷的时刻点
else
    % 前一时刻载荷为极值, 当前时刻载荷的时刻点导入各风机极值存储库

marker2{j}(number2{j}+1)=(-1)^(marker2{j}(number2{j})-1)+marker2{j}(number2{j});
    initial_data2{j}(marker2{j}(number2{j}+1),number2{j}+1)=data2(i,j); %当
    前时刻载荷的时刻点导入各风机极值存储库

```

```

        initial_data2{j}(3,number2{j}+1)=i; %当前时刻载荷的时刻点
        number2{j}=number2{j}+1; %各风机极值存储库计数
    end

    % 各风机极值存储库最新载荷是否替换过
    if judge2{j}==0
        %未替换过，各风机极值存储库的最新载荷导入各风机算法预存库

        realrainflow_data2{j}(1,end+1)=initial_data2{j}(marker2{j}(number2{j}),number2{j})
        ; %各风机极值存储库最新载荷导入各风机算法预存库
        realrainflow_data2{j}(2,end)=initial_data2{j}(3,number2{j}); %各风机算法
        预存库里最新载荷为哪一时刻的载荷

        realrainflow_data2_number{j}(1)=size(realrainflow_data2{j}(1,1:end),2); %各风机算法
        预存库载荷数量
    else
        %替换过，各风机极值存储库的最新载荷替换各风机算法预存库的最新载荷
        judge2{j}=0; %重置判断标记

        realrainflow_data2{j}(1,end)=initial_data2{j}(marker2{j}(number2{j}),number2{j});
        %各风机极值存储库的最新载荷替换各风机算法预存库的最新载荷
        realrainflow_data2{j}(2,end)=initial_data2{j}(3,number2{j}); %各风机算法
        预存库里最新载荷为哪一时刻的载荷
    end

    % 判断各风机算法预存库里载荷数量是否大于等于 3 个
    while realrainflow_data2_number{j}(1)>=3
        % 各风机算法预存库最新 3 个载荷作为实时雨流计数法三点数据
        u2{j}=realrainflow_data2{j}(1,end-2);
        v2{j}=realrainflow_data2{j}(1,end-1);
        w2{j}=realrainflow_data2{j}(1,end);
        % 比较相邻两个点范围大小
        if abs(v2{j}-u2{j})<abs(v2{j}-w2{j})
            cycle2number{j}(1)=cycle2number{j}(1)+1; %各风机实际载荷循环计数
            % 各风机算法预存库载荷数量是否等于 3，进行全循环半循环判断
            if realrainflow_data2_number{j}(1)==3
                cycle2{j}(cycle2number{j}(1),1)=0.5;
                amplitude2{j}(cycle2number{j}(1),1)=abs(v2{j}-u2{j}); %各风机实际
                载荷循环幅值
                meanvalue2{j}(cycle2number{j}(1),1)=(u2{j}+v2{j})/2; %各风机实际
                载荷循环均值
                valueradius2{j}(cycle2number{j}(1),1)=realrainflow_data2{j}(2,end-2); %各风机实际载

```

荷循环起点时刻

valueradius2{j}(cycle2number{j}(1),2)=realrainflow_data2{j}(2,end-1); %各风机实际载
荷循环终点时刻

realrainflow_data2{j}(:,end-2)=[];

realrainflow_data2_number{j}(1)=size(realrainflow_data2{j}(1,1:end),2); %算法预存库
里载荷数量

else

cycle2{j}(cycle2number{j}(1),1)=1;

amplitude2{j}(cycle2number{j}(1),1)=abs(v2{j}-u2{j}); %各风机实际

载荷循环幅值

meanvalue2{j}(cycle2number{j}(1),1)=(u2{j}+v2{j})/2; %各风机实际

载荷循环均值

valueradius2{j}(cycle2number{j}(1),1)=realrainflow_data2{j}(2,end-2); %各风机实际载
荷循环起点时刻

valueradius2{j}(cycle2number{j}(1),2)=realrainflow_data2{j}(2,end-1); %各风机实际载
荷循环终点时刻

realrainflow_data2{j}(:,end-2)=[];

realrainflow_data2{j}(:,end-1)=[];

realrainflow_data2_number{j}(1)=size(realrainflow_data2{j}(1,1:end),2); %算法预存库
里载荷数量

end

else

break;

end

end

% 各风机实际载荷循环计算实际累积疲劳损伤

if cycle2number{j}(1)>=1

eqamplitude2{j}(1:cycle2number{j}(1),1)=amplitude2{j}(1:cycle2number{j}(1),1)./(1-
meanvalue2{j}(1:cycle2number{j}(1),1)./50000000); %修正载荷循环,修正为均值为0的载荷循环
幅值

Ln2{j}(i,1)=sum((eqamplitude2{j}(1:cycle2number{j}(1),1).^10).*cycle2{j}(1:cycle2n
umber{j}(1),1),1)./42565440.4361;

N2{j}(1:cycle2number{j}(1),1)=9.77*10^70./(eqamplitude2{j}(1:cycle2number{j}(1),1)
.^10); %S-N 曲线判断同一载荷循环幅值的最大循环载荷次数


```

Df2{j}(i,1)=sum((cycle2{j}(1:cycle2number{j}(1),1)./N2{j}(1:cycle2number{j}(1),1)
).^ (1-(exp(-valueradius2{j}(1:cycle2number{j}(1),1)./5))./40),1); %非线性累积损伤理
论计算实际累积疲劳损伤值
    end

    % 计算最后实际以及暂时载荷循环
    realrainflow_data22{j}=realrainflow_data2{j};
    realrainflow_data2_number{j}(2)=size(realrainflow_data22{j}(1,1:end),2);
    if i==time1
        while realrainflow_data2_number{j}(2)>=2
            cycle2number{j}(2)=cycle2number{j}(2)+1; %载荷循环计数
            cycle2{j}(cycle2number{j}(2),2)=0.5;

amplitude2{j}(cycle2number{j}(2),2)=abs(realrainflow_data22{j}(1,1)-realrainflow_
data22{j}(1,2));

meanvalue2{j}(cycle2number{j}(2),2)=(realrainflow_data22{j}(1,1)+realrainflow_dat
a22{j}(1,2))/2;
            valueradius2{j}(cycle2number{j}(2),3)=realrainflow_data22{j}(2,1);
            valueradius2{j}(cycle2number{j}(2),4)=realrainflow_data22{j}(2,2);
            realrainflow_data22{j}(:,1)=[];

realrainflow_data2_number{j}(2)=size(realrainflow_data22{j}(1,1:end),2);
        end
        % 各风机暂时载荷循环计算暂时累积疲劳损伤
        if cycle2number{j}(2)>=1

eqamplitude2{j}(1:cycle2number{j}(2),2)=amplitude2{j}(1:cycle2number{j}(2),2)./(1
-meanvalue2{j}(1:cycle2number{j}(2),2)./50000000); %修正载荷循环，修正为均值为0的载
荷循环幅值

Ln2{j}(i,2)=sum((eqamplitude2{j}(1:cycle2number{j}(2),2).^10).*cycle2{j}(1:cycle2
number{j}(2),2),1)./42565440.4361;

N2{j}(1:cycle2number{j}(2),2)=9.77*10^70./(eqamplitude2{j}(1:cycle2number{j}(2),2
).^10); %S-N 曲线判断同一载荷循环幅值的最大循环载荷次数

Df2{j}(i,2)=sum((cycle2{j}(1:cycle2number{j}(2),2)./N2{j}(1:cycle2number{j}(2),2)
).^ (1-(exp(-valueradius2{j}(1:cycle2number{j}(2),3)./5))./40),1); %非线性累积损伤理
论计算暂时累积疲劳损伤值
        end
    else
        while realrainflow_data2_number{j}(2)>=2

```

```

        cycle2number{j}(2)=cycle2number{j}(2)+1; %载荷循环计数
        cycle2{j}(cycle2number{j}(2),2)=0.5;

amplitude2{j}(cycle2number{j}(2),2)=abs(realrainflow_data22{j}(1,1)-realrainflow_
data22{j}(1,2));

meanvalue2{j}(cycle2number{j}(2),2)=(realrainflow_data22{j}(1,1)+realrainflow_dat
a22{j}(1,2))/2;
        valueradius2{j}(cycle2number{j}(2),3)=realrainflow_data22{j}(2,1);
        valueradius2{j}(cycle2number{j}(2),4)=realrainflow_data22{j}(2,2);
        realrainflow_data22{j}(:,1)=[];

realrainflow_data2_number{j}(2)=size(realrainflow_data22{j}(1,1:end),2);
    end
    % 各风机暂时载荷循环计算暂时累积疲劳损伤
    if cycle2number{j}(2)>=1

eqamplitude2{j}(1:cycle2number{j}(2),2)=amplitude2{j}(1:cycle2number{j}(2),2)./(1
-meanvalue2{j}(1:cycle2number{j}(2),2)./50000000); %修正载荷循环，修正为均值为0的载
荷循环幅值

Ln2{j}(i,2)=sum((eqamplitude2{j}(1:cycle2number{j}(2),2).^10).*cycle2{j}(1:cycle2
number{j}(2),2),1)./42565440.4361;

N2{j}(1:cycle2number{j}(2),2)=9.77*10^70./(eqamplitude2{j}(1:cycle2number{j}(2),2
).^10); %S-N 曲线判断同一载荷循环幅值的最大循环载荷次数

Df2{j}(i,2)=sum((cycle2{j}(2:cycle2number{j}(2),2)./N2{j}(2:cycle2number{j}(2),2)
),1)+(cycle2{j}(1,2)./N2{j}(1,2)).^(1-(exp(-valueradius2{j}(1,3)./5))./40); %非线性
累积损伤理论计算暂时累积疲劳损伤值
    end
    realrainflow_data22{j}=[];
    amplitude2{j}(1:cycle2number{j}(2),2)=0;
    meanvalue2{j}(1:cycle2number{j}(2),2)=0;
    valueradius2{j}(1:cycle2number{j}(2),3)=0;
    valueradius2{j}(1:cycle2number{j}(2),4)=0;
    cycle2number{j}(2)=0;
    end
    % 计算总累积疲劳损伤
    if cycle2number{j}(1)>=1
        Lnall2(i,j)=(Ln2{j}(i,1)+Ln2{j}(i,2))^(1/10);
        Dfall2(i,j)=Df2{j}(i,1)+Df2{j}(i,2);
    else

```

```
        Lnall2(i,j)=(Ln2{j}(i,2))^(1/10);  
        Dfall2(i,j)=Df2{j}(i,2);  
    end  
  
end  
  
end  
toc;
```

Python 代码:

主轴扭矩和塔架推力的估计模型程序:

```
% 公式  $0.98444 \cdot P_{ref\_t-1} + 4.883V^3 + 45178.9152$ 
data=load('附件 2-风电机组采集数据.mat');
WT=data.data_TS_WF.WF_1.WT;
T_N_all=[];
T_push_all=[];
T_N_error_all=[];
T_push_error_all=[];
T_N_error_rel_all=[];
T_push_error_rel_all=[];
for ID =1:100
    WF_id=WT{ID};
    input = WF_id.inputs;
    output = WF_id.outputs;
    state = WF_id.states;
    R= 65; % 风轮半径
    Rho =1.205; % 空气密度
    % 提取数据
    V = input(2:2000,2);
    P_ref = input(1:1999,1);
    % 拟合公式
    X = [V, P_ref]; % 直接将 V 和 P_ref 组合成一个矩阵
    Y = output(2:2000,3);
    % 定义模型函数
    modelFun = @(K, X) K(1)*X(:,2) + K(2)*X(:,1).^3+K(3);
    % 初始参数猜测
    initialGuess = [1, 0,0]; % 初始猜测 K 的值
    % 设置优化选项
    options = optimoptions('lsqcurvefit', 'Algorithm', 'levenberg-marquardt');
    % 使用 lsqcurvefit 进行拟合
    [fittedParams, resnorm, residual, exitflag, out] = lsqcurvefit(modelFun, initialGuess, X, Y, [], [], options);
    k1=fittedParams(1);
    k2=fittedParams(2);
    k3=fittedParams(3);

    % 初始第一个
    P_out_1 = k1*input(1,1)+k2*input(1,2)^3+k3;
    P_out = k1*P_ref+k2*V.^3+k3;
    P_out_all=[P_out_1;P_out];
    V_all = input(:,2);
```

```

w_m = state(:,2); % 风轮转速
P_wind = 0.5*pi*Rho*R^2*V_all.^3;
P_ref_all = input(:,1);
C_P = P_out_all/P_wind;
a = zeros(size(C_P));
for i = 1:length(C_P)
    % 定义一个匿名函数
    f = @(a) 4*a*(1-a)^2 - C_P(i);
    % 使用 fzero, 需要提供一个好的初始猜测值
    % 这里我们尝试两个初始猜测值, 选择更接近 1 的那个解 (如果存在)
    a_guess1 = 0.1;
    a_guess2 = 0.9;
    a1 = fzero(f, a_guess1);
    a2 = fzero(f, a_guess2);

    % 检查哪个解在(0,1)内并且是较大的
    valid_sols = [a1, a2];
    valid_sols = valid_sols(valid_sols >= 0 & valid_sols <= 1);
    if ~isempty(valid_sols)
        a(i) = max(valid_sols);
    else
        a(i) = 0.5; % 如果没有解
    end
end
end
T_N = P_out_all./w_m;
T_push = P_out_all./(V_all.*(1-a));
T_push_error = T_push-output(:,2);
T_push_error_rel = (T_push-output(:,2))./(output(:,2));
T_N_error = T_N-output(:,1);
T_N_error_rel = (T_N-output(:,1))./(output(:,1));
T_push_all=[T_push_all,T_push];
T_N_all=[T_N_all,T_N];
T_push_error_all=[T_push_error_all,T_push_error];
T_N_error_all=[T_N_error_all,T_N_error];
T_push_error_rel_all=[T_push_error_rel_all,T_push_error_rel];
T_N_error_rel_all=[T_N_error_rel_all,T_N_error_rel];
end
% 写入第二个矩阵到第二个工作表
xlswrite('result.xlsx', T_N_all, 'T_N');
xlswrite('result.xlsx', T_push_all, 'T_push');
xlswrite('result.xlsx', T_N_error_all, 'T_N_error');
xlswrite('result.xlsx', T_push_error_all, 'T_push_error');
xlswrite('result.xlsx', T_N_error_rel_all, 'T_N_error_rel');

```

```
xlsxwrite('result.xlsx', T_push_error_rel_all, 'T_push_error_rel');
```

强化学习风电场环境建模程序:

```
import random
import matlab.engine
import matlab
import numpy as np
import pygame
import gymnasium as gym
from gymnasium import spaces
from scipy.stats import beta
import torch
import networkx as nx
from pypower.api import runpf, poption
import pandas as pd

class PowerSystemEnv(gym.Env):
    metadata = {"render_modes": ["human", "rgb_array"], "render_fps": 20}

    def __init__(self, render_mode=None):
        # 观测值包括 100 个风机的当前累计损耗, 100 个风速, 当前所需要的总功率
        self.observation_space = spaces.Box(np.zeros(201), np.ones(201), dtype=np.float64)
        self.action_space = spaces.Box(np.zeros(100), np.ones(100), dtype=np.float64)
        # 初始化疲劳损伤初值, 初始风速(取 15m/s), 根据时刻实时更新
        self.reward=0
        self.P=388
        self.D=np.zeros(100)
        self.wind = 15*np.ones(100)
        self.max_last = 0
        self.P_last=3*np.ones(100)
        self.t = 0
        self.n = 0
        self.flag =0
        self.info = { }
        self.data=np.array(pd.read_excel('Data.xlsx', header=None))
        # 归一化设定值, 用以加快模型的拟合速度
        self.P_all_max = 500 #单位 MW
        self.P_all_min = 300
        self.P_max = 5 #单位 MW
        self.P_min = 3
        self.V_max = 16
```

```

        self.V_min = 13
        self.eng = matlab.engine.start_matlab() #初始化 matlab 引擎
        assert render_mode is None or render_mode in
self.metadata["render_modes"]
        self.render_mode = render_mode
        self.window = None
        self.clock = None
        self.window_size = 600

    def seed(self, seed):
        np.random.seed(seed)

    def _get_obs(self):
        # 观测状态给出前三时段的新能源发电功率/负荷功率/电价与当前储能容量
        # 此处进行数据的归一化, 环境中计算使用实际值但传输给神经网络以及神经网络传输到环境的均为归一化后的数值(0, 1)
        P_all = (self.P-self.P_all_min)/(self.P_all_max-self.P_all_min)
        D = self.D
        V = (self.wind-self.V_min)/(self.V_max-self.V_min)
        obs =
np.concatenate((np.array(P_all).reshape(-1, ), D.reshape(-1, ), V ), axis=0)
        return obs

    def _get_info(self):
        info = self.info
        return info

    def reset(self, seed=None, options=None):
        if seed is None:
            seed = random.randint(1, 200)
        super().reset(seed = seed)
        self.P=388
        self.reward=0
        self.D=np.zeros(100)
        self.wind = 15*np.ones(100)
        self.max_last = 0
        self.P_last=3*np.ones(100)
        self.t = 0
        self.flag = 0
        self.info = {}
        observation = self._get_obs()
        info = self._get_info()
        if self.render_mode == "human":

```

```

        self._render_frame()
    return observation, info

def compute_S(self, P_last, wind):
    # 通过当前决策动作和风速，使用问题 2 所得拟合模型计算载荷
    # action 形状(100, 1) 风速形状(100, 1)
    P_out=0.985*10**6*P_last+59150
    S_N= P_out/1.25 #取平均转速
    S_push=
10**6*(-14.62)/P_out+7512*0.38*wind**2+0.18*P_out/wind-450000
    return S_N, S_push

def step(self, action):
    self.n=self.n+1
    truncated = False
    terminated = False
    if self.t==99: #完成一个任务回合，重置
        truncated = True
        self.flag = 1
    else:
        self.flag =0
    # 对归一化后的功率动作进行反归一化
    P_now = action*(self.P_max-self.P_min)+self.P_min
    # 计算奖励(疲劳损伤增量)
    S_N, S_push=self.compute_S(self.P_last, self.wind)
    # 与 matlab 交互根据问题 1 所得的实时雨流法，计算综合疲劳损伤，输出机
组最大的疲劳损耗增量(0 或增量)
    # print('给的载荷', S_N[0], S_push[0])
    D_now = self.eng.realrainflow3(S_N, S_push, self.flag, self.n)
    D_now=np.array(D_now)
    D_sum=np.sum(D_now)
    self.reward = 1-1*1e10*(D_sum-self.max_last)/100
    self.max_last = D_sum
    if self.t!=99:
        self.t=self.t+1
        self.D=D_now*1e10
        self.P_last = P_now
        self.wind= self.data[self.t, 1:]
        self.P = self.data[self.t, 0]/1e6
    observation_next = self._get_obs()
    info = {'D': D_now}

```



```

        if self.render_mode == "human":
            self._render_frame()
        return observation_next, self.reward, terminated, truncated, info

    def render(self):
        if self.render_mode == "rgb_array":
            return self._render_frame()

    def _render_frame(self):
        return None

    def close(self):
        if self.window is not None:
            pygame.display.quit()
            pygame.quit()

```

强化学习功率分配决策模型参数训练:

```

import gymnasium as gym
import torch, numpy as np, torch.nn as nn
from torch.utils.tensorboard import SummaryWriter
import tianshou as ts
from tianshou.exploration import GaussianNoise
from tianshou.utils.net.common import Net
import gym_examples
task = 'gym_examples/Power_system-v0'
lr, epoch, batch_size = 1e-4, 5, 64
buffer_size = 20000
step_per_epoch, step_per_collect = 2000, 8
gamma=0.99

test_num=2
logger = ts.utils.TensorboardLogger(SummaryWriter('../log/ddpg'))
#test
train_envs = ts.env.DummyVectorEnv([lambda: gym.make(task)])
test_envs = ts.env.DummyVectorEnv([lambda: gym.make(task)])

class ActorNet(nn.Module):
    def __init__(self, hidden_dim, output_dim):
        # 输入 10 输出 2
        super(ActorNet, self).__init__()
        self.fc1 = nn.Linear(201, 2*hidden_dim)
        self.fc2 = nn.Linear(2*hidden_dim, hidden_dim)

```

```

self.fc3 = nn.Linear(hidden_dim, output_dim)
self.relu = nn.LeakyReLU()
self.sigmoid=nn.Sigmoid()
self.relu1 =nn.ReLU()
self.tanh=nn.Tanh()
self.softmax=nn.Softmax()

def forward(self, obs, state=None, info={}):
    if not isinstance(obs, torch.Tensor):
        obs = torch.tensor(obs, dtype=torch.float)
        input = self.fc1(obs)
        input = self.relu(input)
        input = self.fc2(input)
        input = self.relu(input)
        input = self.fc3(input)
        input = self.sigmoid(input)*0.5+0.725
        #print(input[0])
        scale = (input/(input.sum(dim=1, keepdim=True)+1e-9))
        scale = scale.clamp(min=0.007, max=0.013)
        scale_real =
scale.reshape(-1,100)*(obs[:,0].reshape(-1,1)*200+300) # 功率总值按 300-500
归一化
        # 将每台风机功率再次进行归一化,这个数据将传输到 critic 网络,因此
同样需要进行归一化以提升参数的拟合速度
        out = (scale_real - 3)/2
    return out, state

class CriticNet(nn.Module):
    def __init__(self, hidden_dim, output_dim):
        #observation, 3 action 1
        super(CriticNet, self).__init__()
        self.fc1 = nn.Linear(301, 2*hidden_dim)
        self.fc2 = nn.Linear(2*hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
        self.relu = nn.LeakyReLU()

    def forward(self, obs, act, state=None, info={}):
        obs = torch.tensor(obs).squeeze().float()
        if not isinstance(act, torch.Tensor):
            act = torch.tensor(act)
        input= torch.cat((obs, act), dim=1)
        input = self.fc1(input.float())

```

```

        input = self.relu(input)
        input = self.fc2(input)
        input = self.relu(input)
        Q = self.fc3(input)
        return Q

#torch.autograd.set_detect_anomaly(True)
ActorNet = ActorNet(1024, 100)
Criticnet1 = CriticNet(1024, 1)
Criticnet2 = CriticNet(1024, 1)
A_optim = torch.optim.Adam(ActorNet.parameters(), lr=lr)
C_optim1 = torch.optim.Adam(Criticnet1.parameters(), lr=10*lr)
C_optim2 = torch.optim.Adam(Criticnet2.parameters(), lr=10*lr)
policy = ts.policy.TD3Policy(
    actor=ActorNet,
    actor_optim=A_optim,
    critic1=Criticnet1,
    critic1_optim=C_optim1,
    critic2=Criticnet2,
    critic2_optim=C_optim2,
    gamma=gamma,
    tau=0.005,
    estimation_step=2)
train_collector = ts.data.Collector(policy, train_envs,
ts.data.ReplayBuffer(buffer_size), exploration_noise=True)
test_collector = ts.data.Collector(policy, test_envs,
exploration_noise=False)
result = ts.trainer.OffpolicyTrainer(
    policy=policy,
    train_collector=train_collector,
    test_collector=test_collector,
    max_epoch=epoch,
    step_per_epoch=step_per_epoch,
    step_per_collect=step_per_collect,
    episode_per_test=test_num,
    batch_size=batch_size,
    update_per_step=1 / step_per_collect,
    logger=logger,
    repeat_per_collect=1
).run()

print(f'Finished training! Use {result["duration"]}')
torch.save(policy.state_dict(), 'ddpg.pth')

```

```
policy.load_state_dict(torch.load('ddpg.pth'))
```

考虑采集数据不确定性的优化模型建模程序：

```
# import gurobipy
from gurobipy import Model, GRB
import pandas as pd
#读取数据
k10 = pd.read_excel('D:\pyproject\数据驱动\model_found\Para.xlsx',sheet_name='k1',header=None)
k20=pd.read_excel('D:\pyproject\数据驱动\model_found\Para.xlsx',sheet_name='k2',header=None)
k30=pd.read_excel('D:\pyproject\数据驱动\model_found\Para.xlsx',sheet_name='k3',header=None)
PV=pd.read_excel('D:\pyproject\数据驱动\model_found\PV.xlsx',header=None)
PV = PV.to_numpy()
k10 = k10.to_numpy()
k20 = k20.to_numpy()
k30 = k30.to_numpy()
P0=PV[0,:]
V0=PV[1:100,:]
print(k10.shape)
# print(P0)
print(V0[0,99])
print(k10[0,99])
print(k20[0,99])
print(k30[0,99])
# 创建一个新的模型
m = Model("prod_mix")
num_vars=100
k1=10
k2=3
v=10#风速
k3=1
Mtb=20000000#基准转矩
Mfb=2000000#基准推力
P0=100#总功率
Pmin=0#机组最大功率
Pmax=4000000#机组最小功率
# 定义决策变量
Pref = [m.addVar(vtype=GRB.CONTINUOUS, name=f"x_{i}") for i in range(100)]#功率
Mt = [m.addVar(vtype=GRB.CONTINUOUS, name=f"x_{i}") for i in range(100)]#主轴转矩
Mf = [m.addVar(vtype=GRB.CONTINUOUS, name=f"x_{i}") for i in range(100)]#塔架推力
```